LegatorOCR – User guide

Edition 0.2.0-pre.61 (trial version). Work in progress; this English edition is translated from the Romanian original. Where a caption says the interface is in Romanian, the screenshot shows the Romanian interface.

Zoom: in the bottom bar, **A+** / **A-** enlarge or shrink the whole interface (also Ctrl + / Ctrl −), and **Auto** fits it to the window size. The window starts maximised with an automatic zoom, so the text is large and fills the screen; your choice is kept.

1. What LegatorOCR is

LegatorOCR turns scans or photos of pages into a clean PDF with searchable text. It solves the same problem as ScanTailor (splitting pages, deskewing, cropping, cleaning), but adds PDF export and OCR. It is written in Rust, with its own code, each stage being a textbook technique. It runs locally, with no account and without sending the pages over the internet. Licence: MIT.

There are two ways of working, with the same engines underneath:

- **The graphical interface** (`legatorocr-gui`): you add pages, see the original and the result side by side, correct page by page and export.
- **The command line** (`legatorocr`): for large batches, for automation and for options the interface does not show.

What it does

- splits double-page sheets into two pages, straightens skewed pages and crops the margins;
- cleans the background, stains, punch holes, fingers on the edge and form lines;
- writes PDF (including PDF/A for archiving) or images, with OCR in Romanian and 7 other languages;
- also produces plain text (`.txt`) and hOCR, and can sign or date-stamp the PDF.

What it does not do (or has not been proven to do)

- It is a trial version (0.2.0-pre.61). It has been run by the author and his assistant only on Linux; **Windows, macOS and arm64 processors have not been tried by people**.
- OCR accuracy was measured on 8 good pages from a single Romanian magazine, against a reference text produced by another program, not corrected by a person. Do not take it as a guarantee for other kinds of print.
- OCR can make mistakes: spot-check the important pages.

2. Installation and first run

Download the archive for your system from the releases page (Releases) of the `chiuta/LegatorOCR` repository. Each archive has a `.sha256` file for verification.

System	File	Notes
Linux x86-64 or ARM64	`.tar.gz`	needs OpenGL 2+ and X11 or Wayland
Windows x86-64	`.zip`	needs OpenGL 2+; it may be missing in plain RDP sessions
macOS Apple-silicon or Intel	`.zip` with `LegatorOCR.app`	**unsigned**: at first launch right-click → Open, or `xattr -dr com.apple.quarantine LegatorOCR.app`

You can also build from source: `cargo build --release` (requires Rust 1.95 or newer).

OCR

The program has its own OCR engine, which works without installation. Optionally you can install Tesseract, a separate program, which LegatorOCR uses automatically when it is present and has the requested language:

- Linux (Debian/Ubuntu): `sudo apt install tesseract-ocr tesseract-ocr-ron`
- macOS: `brew install tesseract tesseract-lang`
- Windows: `winget install UB-Mannheim.TesseractOCR` and adding it to `PATH`

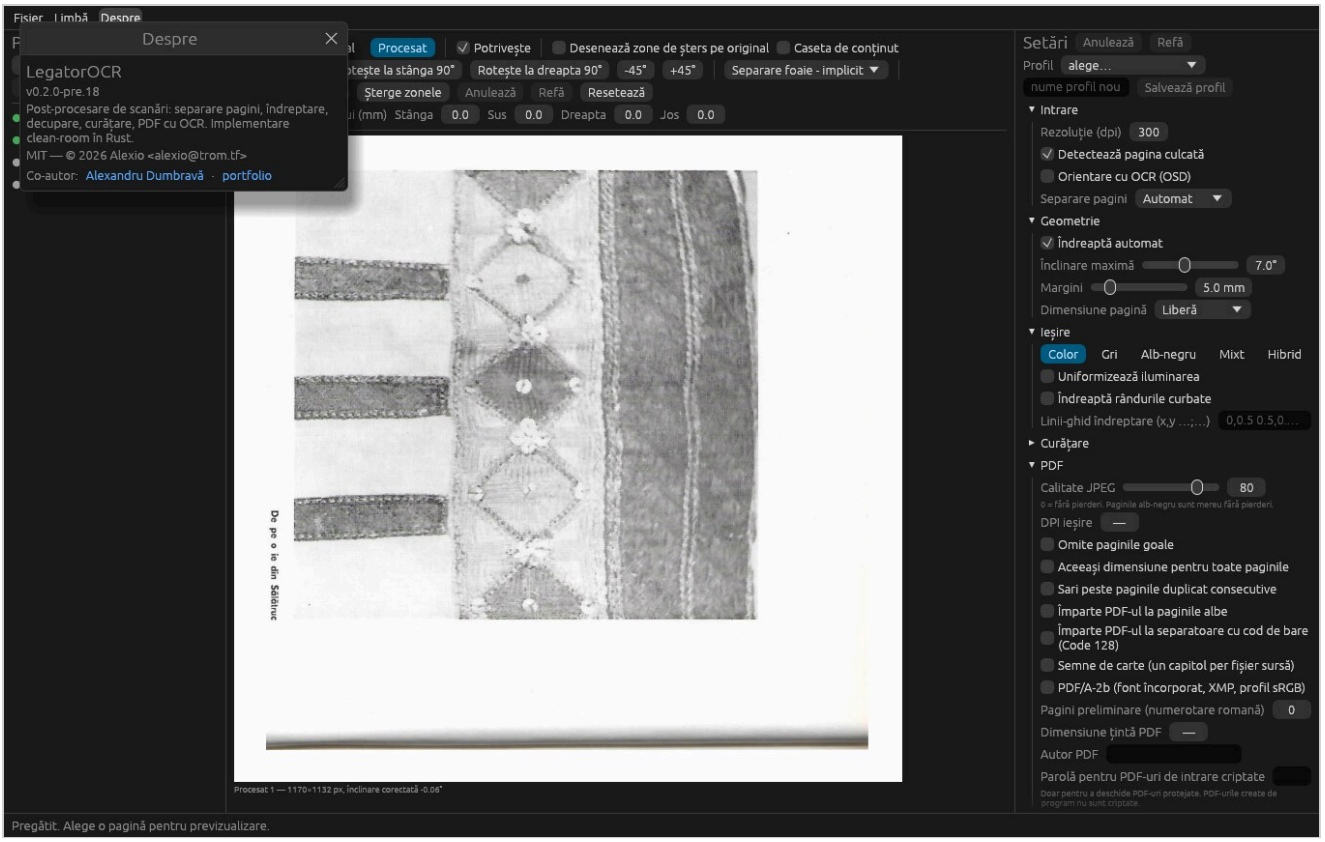
The Romanian model in the `apt` and `brew` packages is the small ("fast") variant. The "best" variant is downloaded manually and pointed to with `--tessdata DIR`. Which of them reads your pages better has not been measured.

Check the installation

Run `legatorocr --doctor`. It prints the version, the native OCR engine, Tesseract and the language models, then auxiliary tools (`pdftoppm`, `gs`, `qpdf`, `mutool`). A "missing" result for an optional tool is not an error; the message below it says whether it is needed. If something does not work, copy the whole output of the command into a report (see section 10).

```
$ legatorocr --doctor
[ok     ] legatorocr                        0.2.0-pre.19
[ok     ] native OCR engine                 built in
[ok     ] tesseract                         tesseract 5.3.4
[ok     ] tesseract model ro (Romanian)     installed
[ok     ] tesseract model en (English)      installed
[ok     ] tesseract model fr (French)       installed
[ok     ] tesseract model it (Italian)      installed
[ok     ] tesseract model es (Spanish)      installed
[ok     ] tesseract model pt (Portuguese)   installed
[ok     ] tesseract model de (German)       installed
[ok     ] tesseract model hu (Hungarian)    installed
[ok     ] pdftoppm                          pdftoppm
[missing] mutool                            not found
          -> optional: alternative renderer for those cases
[ok     ] gs                                gs
[ok     ] qpdf                              qpdf
14 check(s) ok, 1 missing. Missing helpers are optional unless a hint says otherwise.
```

The output of the legatorocr --doctor command (pre.20, Linux, in a real terminal; the same for all terminal screenshots in the guide, made with xterm): "missing" for mutool is an optional tool, not an error.



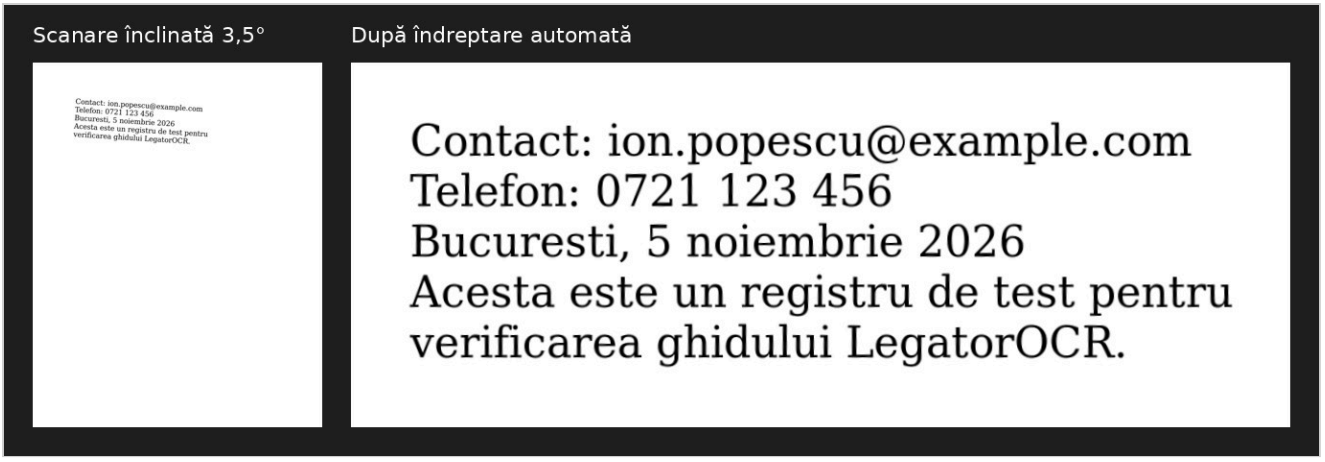
The About menu shows the installed version (here 0.2.0-pre.18). Screenshot from the pre.18 build on Linux, interface in Romanian.

3. The workflow, step by step

For each page, the program goes through the stages below in this order. The analysis is done on a copy of about 150 dpi, and the result is then applied to the scan at its full resolution.

1. **Orientation.** Pages lying on their side (a quarter turn) are detected from the letters of the page: a page with pictures or with few words is not rotated by this test. For the exact direction, Tesseract (OSD) or manual rotation can be used.
2. **Sheet splitting.** A landscape sheet with two pages is cut in the middle, at the empty band in the centre. It can be automatic, at the middle, or off.
3. **Deskewing.** The skew angle is estimated from the text lines and the page is rotated back. The angle can be fixed manually.
4. **Content box.** The area with text and images is found; scanner shadows and dust are ignored. Everything outside it is repainted in the paper colour.
5. **Margins and size.** Margins in millimetres, optionally a fixed page size or the same size for the whole book.
6. **Straightening curved lines** (optional). For dense book pages with lines curved near the spine.
7. **Lighting** (optional). Evens out an uneven background.

8. **Cleaning and binarization.** Removes stains, punch holes, fingers, lines; then black and white, grey, colour, mixed or hybrid.
9. **PDF and OCR.** The PDF (or the images) is written and, if ticked, the text layer.



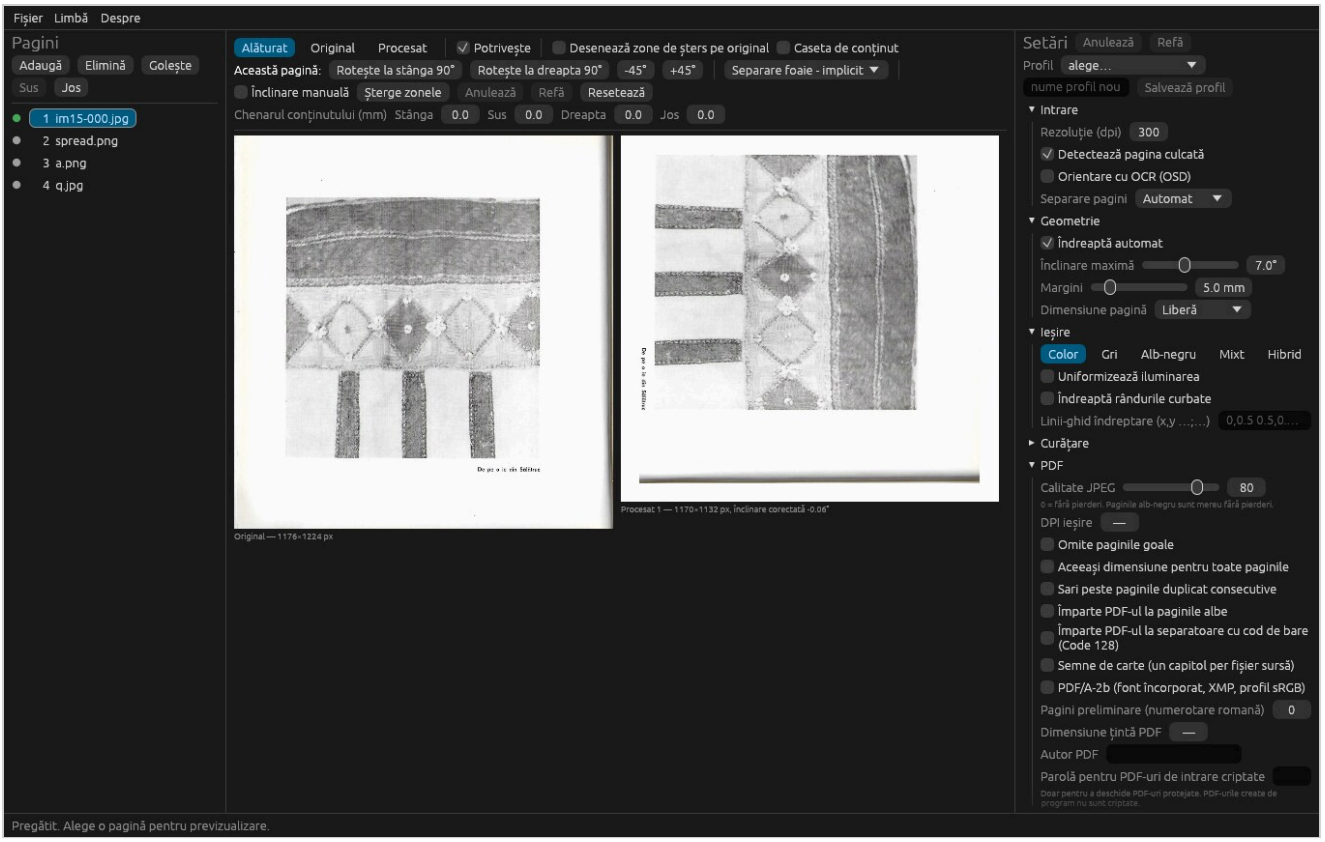
Automatic deskewing: a scan skewed by 3.5° (a test sheet generated by me) and the result; the program measured -3.50°.

Working rules that save time

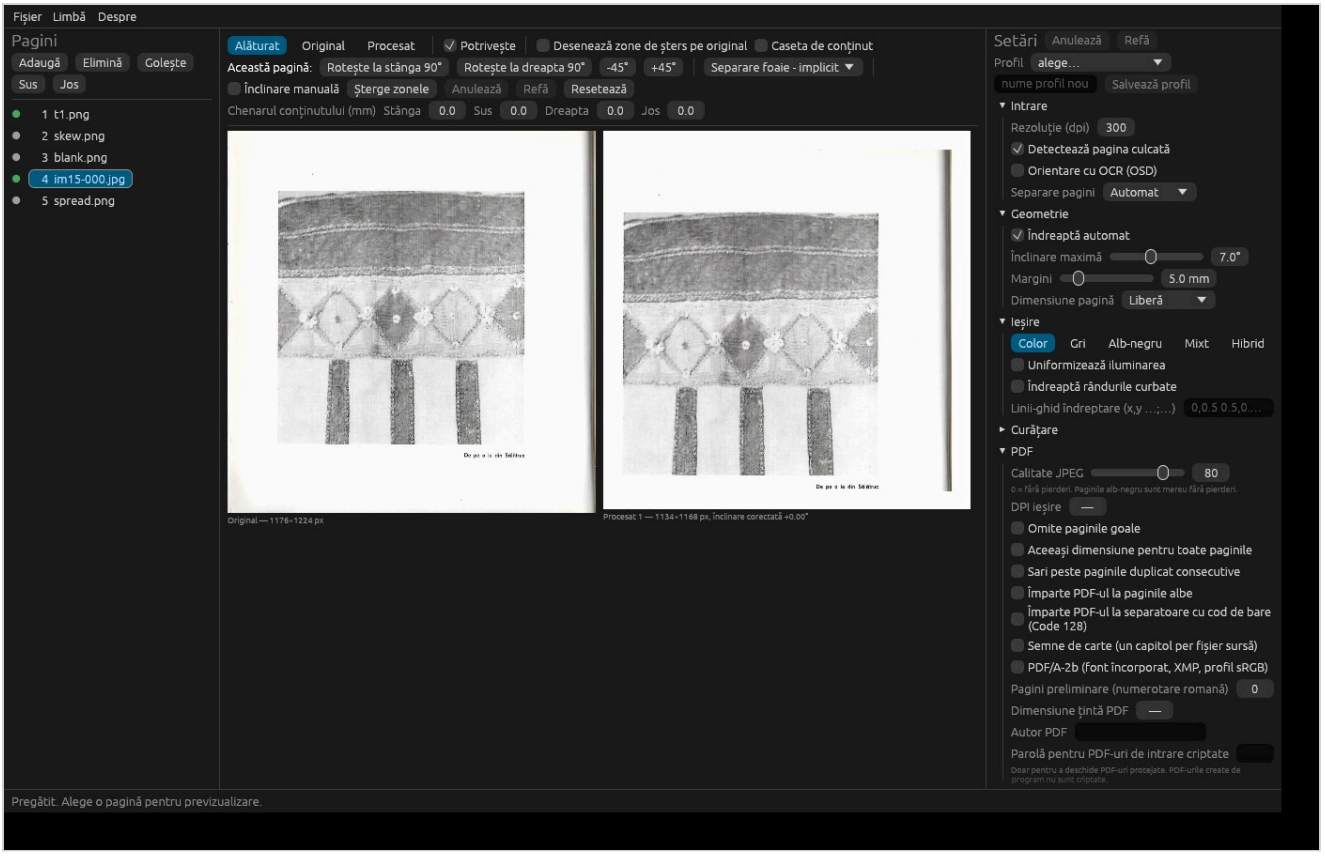
- First set the correct **resolution** (usually 300 dpi for text). The program reads it from the image if it is between 150 and 1200; otherwise it assumes 300.
- Choose a **profile** suited to the material (section 6) before touching individual settings.
- Process 10–20 pages as a trial, check the result, then start the large batch.
- The interface marks pages "to check" (blank page, borderline skew, nothing found, content stuck to the edge). Go through them with the "Next" button.

4. The graphical interface

The window has three areas: the page list on the left, the preview in the centre and the settings on the right. The **File** menu contains "Add images", "Add a folder", "Open project", "Save project", "Export PDF" and "Export images". The interface language (8 languages) is changed from the **Language** menu.



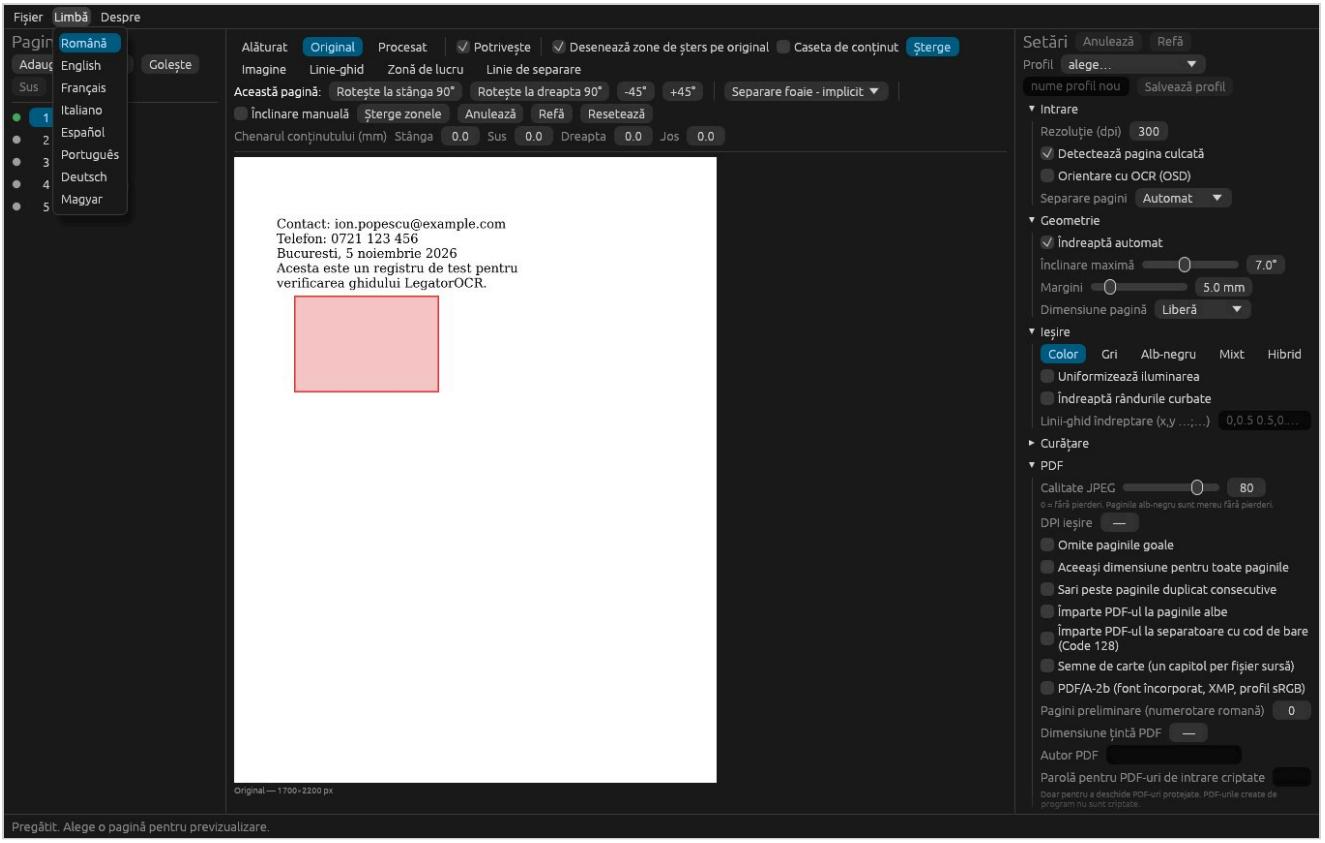
The main window: the page list on the left, the original and the processed page in the middle, the settings on the right. The example page is a photo without text, so the detection of a page lying on its side rotated it wrongly in pre.18 (fixed in pre.19, see below). Screenshot pre.18 on Linux.



The same page in pre.19, with "Detect page lying on its side" ticked: it is no longer rotated (processed 1134×1168 px, skew +0.00°). Screenshot pre.19 on Linux.

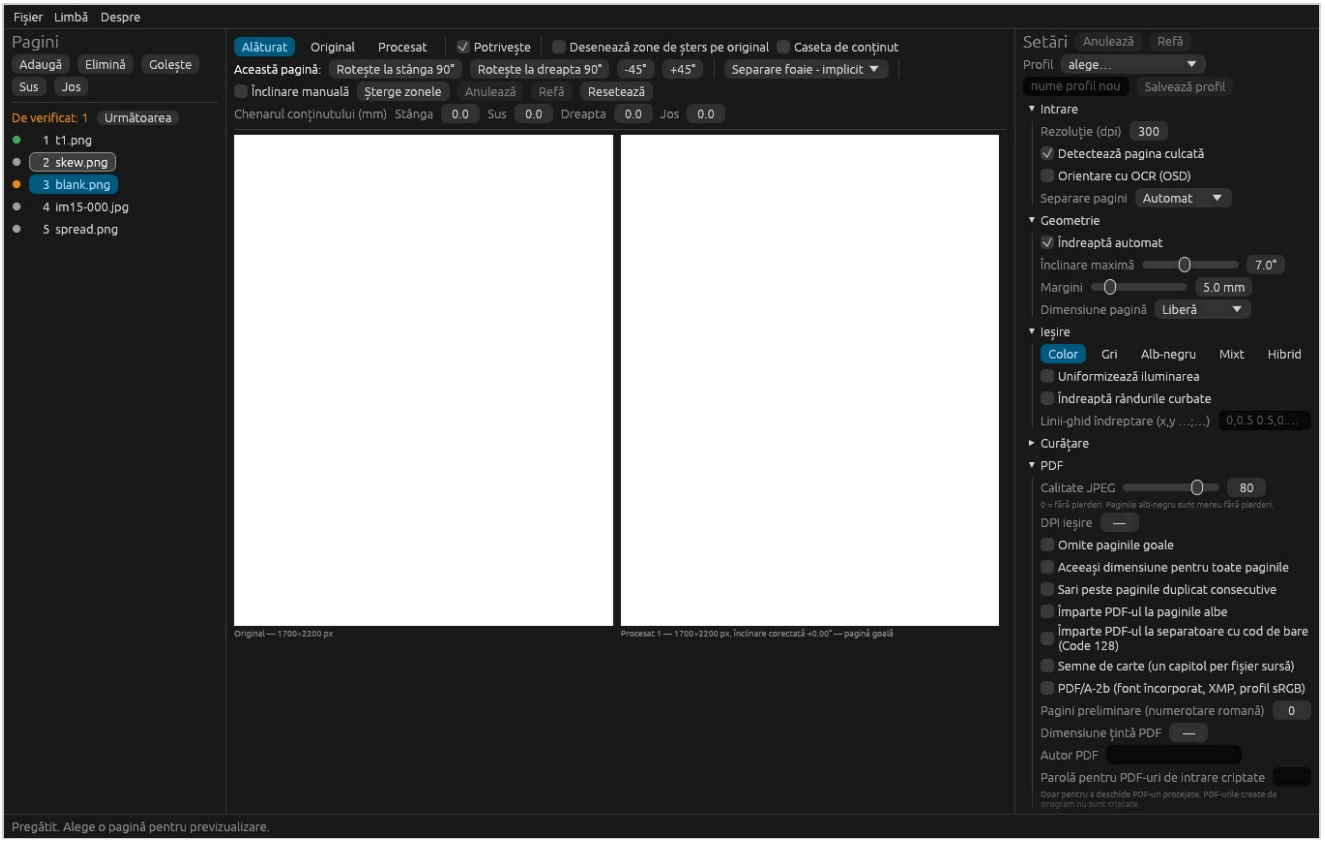


The File menu: adds images or a folder, opens and saves the project, exports PDF or images.



The Language menu: the eight interface languages (Romanian, English, French, Italian, Spanish, Portuguese, German, Hungarian).

The page list

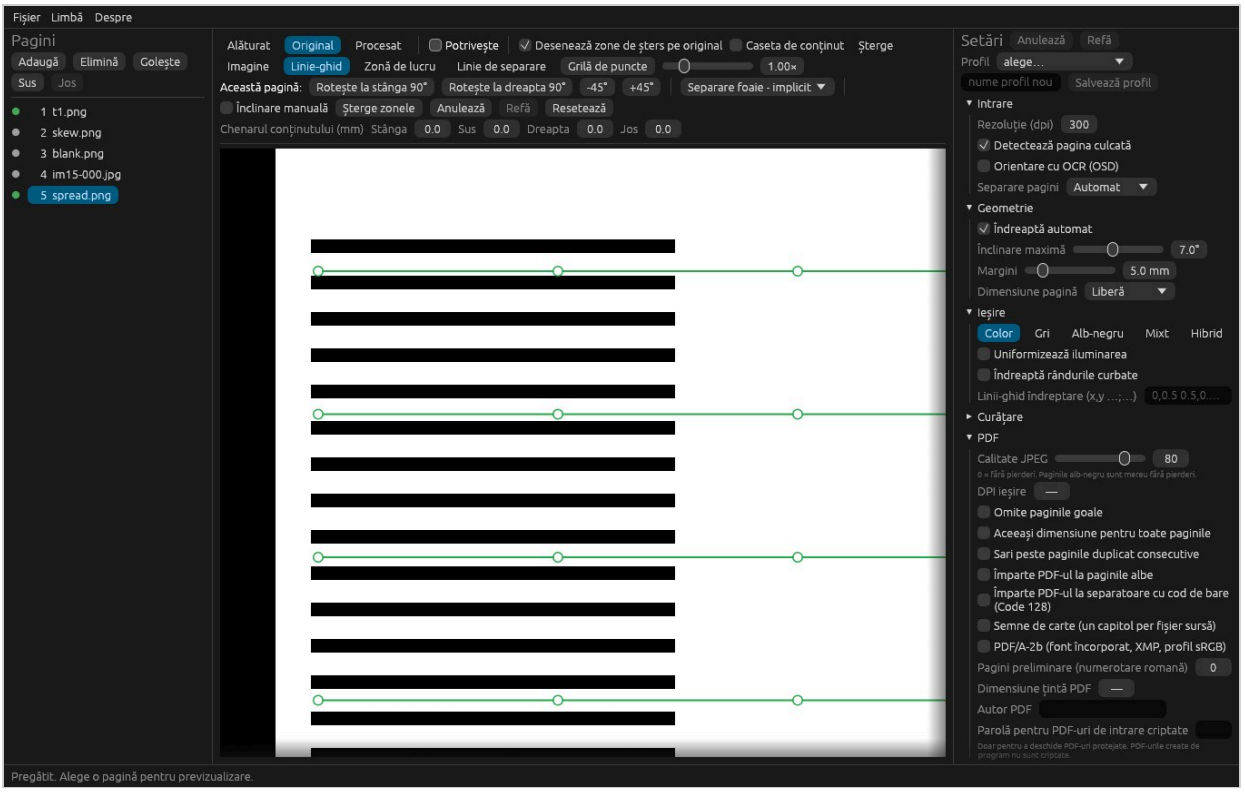


A completely white page is marked automatically: an orange dot in the list and the "To check: 1" panel with the "Next" button.

Drag images, PDFs, archives or a folder into it, or press "Add". The Up/Down buttons change the order, "Remove" takes out the chosen page, "Clear" removes everything. The order in the list is the order in the PDF.

The preview

- **Side by side / Original / Processed** chooses what you see. Zones are drawn on *Original*; the content box is corrected on *Processed*.
- **Fit** enlarges or shrinks the image so that it fits. The zoom slider (0.2×–4×) appears only when *Fit* is unticked.



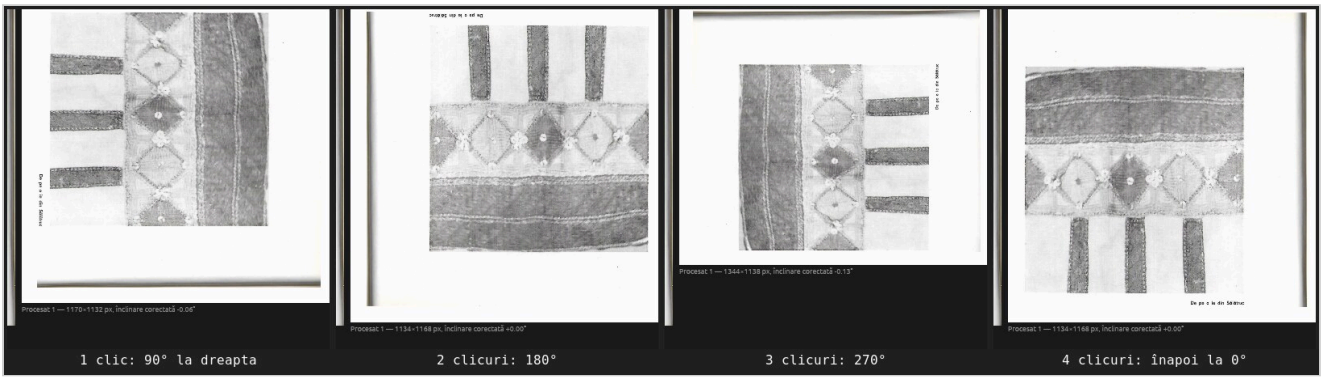
With "Fit" unticked the zoom slider appears (here 1.00×).

Thumbnails (since pre.58). Each page is a card: the small picture on top, the number and name underneath, and the selected page has a blue frame round both. Next to the “Thumbnails” box you choose the size: S (120×90 px), M (240×180, the default) or L (320×240); if the panel is narrower the pictures shrink. Thumbnails are read in the background, only for the pages in view, so a long list does not slow the window. Untick “Thumbnails” for the compact one-line list.

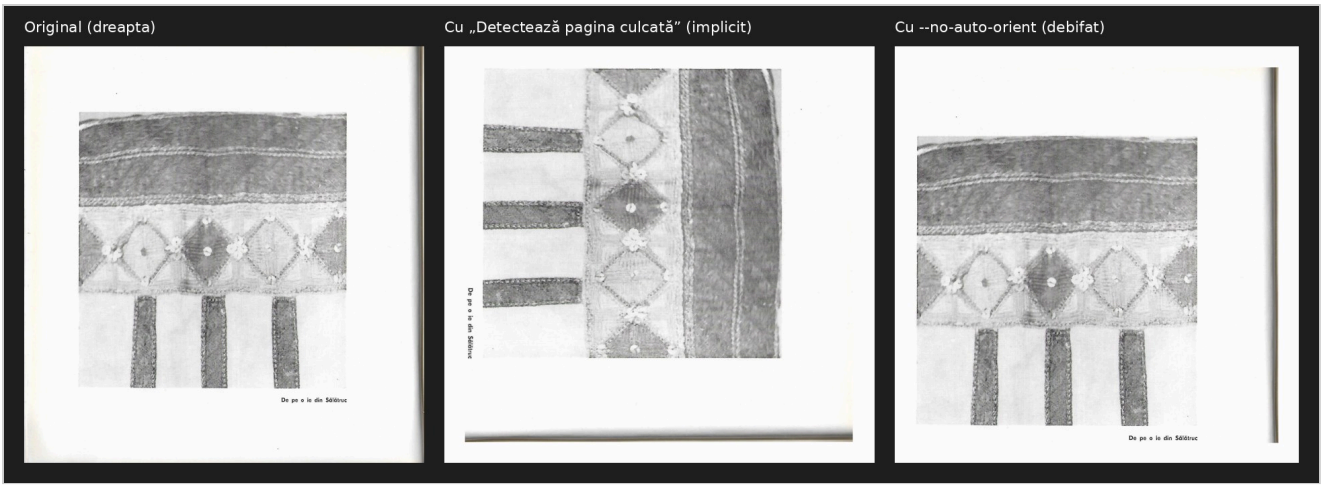
The "This page" bar

Here you correct only the chosen page: rotation left or right by 90°, ±45° steps, manual skew, the sheet-splitting mode (default, automatic, middle, off), the content frame in millimetres (left, top, right, bottom; positive values enlarge it) and "Reset". **Undo** and **Redo** (Ctrl+Z, Ctrl+Y) work for all these commands, not only for zones.

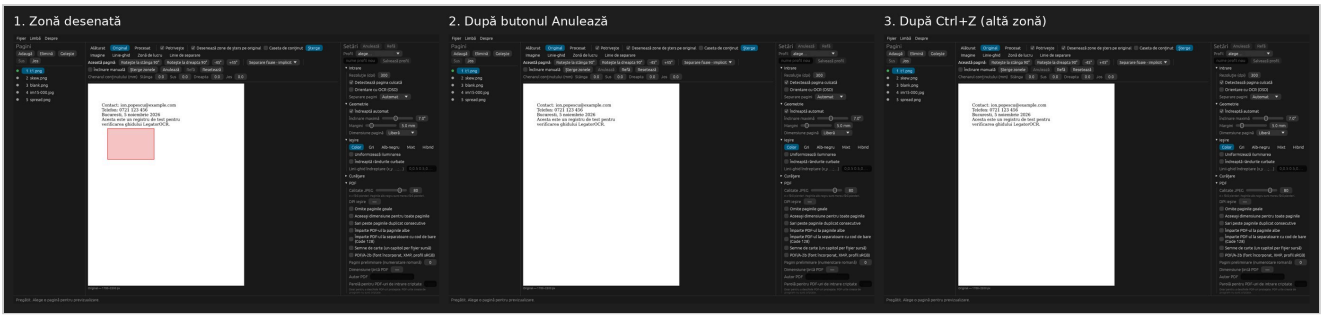
Manual rotation locks the automatic orientation (since pre.18): after you press a rotation button or ±45°, the page is no longer rotated by the detection of a page lying on its side or by OSD, not even at 0°. "Reset" gives the automatic orientation back. If a page with pictures is rotated wrongly from the start, press rotate four times (or untick "Orientation with OCR (OSD)" and "Detect page lying on its side" in the settings, under Input).



Four clicks in a row on "Rotate right 90°": 90°, 180°, 270°, then back to 0°, with no jumps. Screenshot pre.18 on Linux. I could not check on Windows 11.



The same page (page 15 of the tester's PDF) and the same settings, apart from the detection: in pre.18, with "Detect page lying on its side" ticked (the default), the program rotated it by 90° although the original is upright (from pre.19–20 it no longer rotates it); with --no-auto-orient (or unticked in the settings) it comes out upright. Checked on the command line; in the window I saw the rotation in pre.18 and its absence in pre.19.

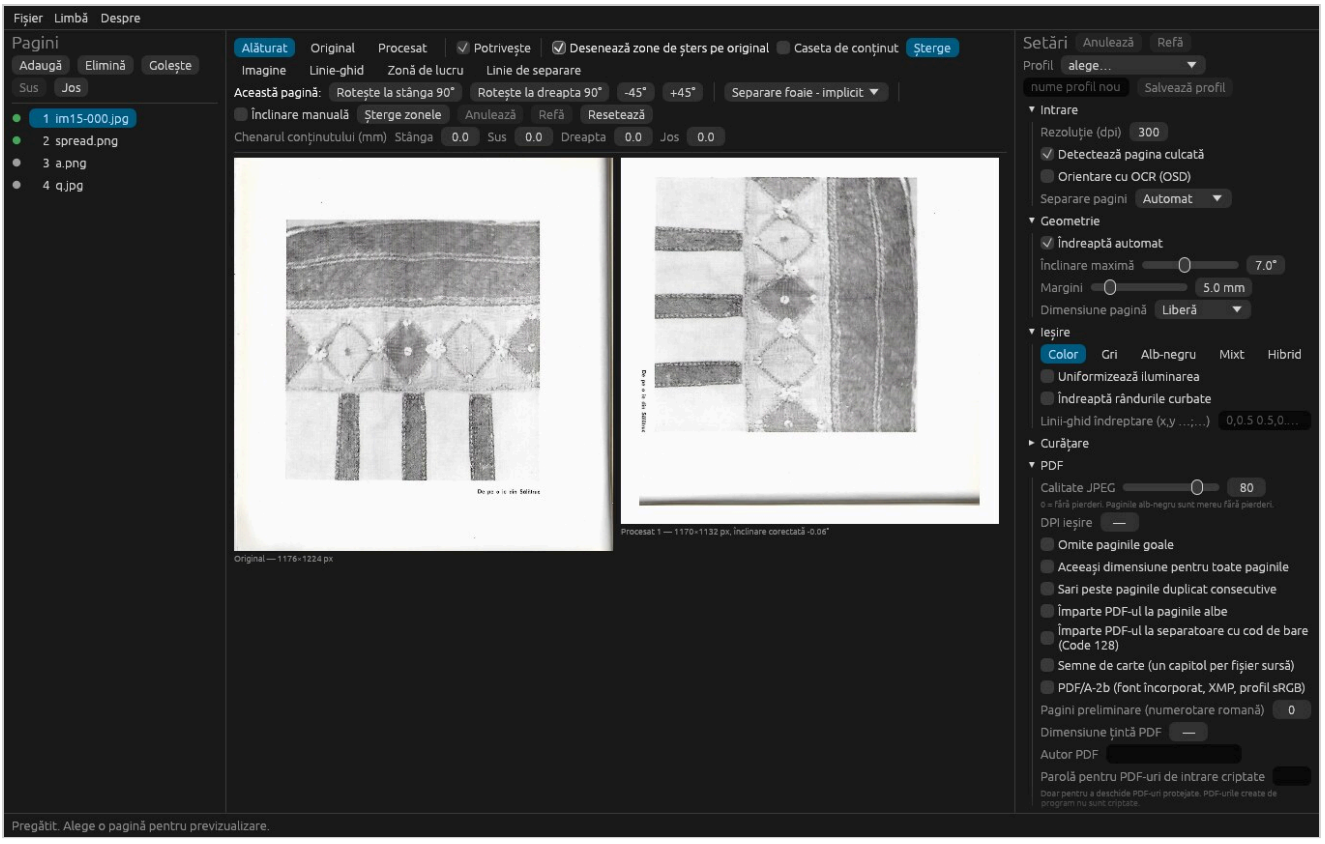


Undo (the button) and Ctrl+Z (with the window in the foreground) remove the last zone drawn. Checked in the window.

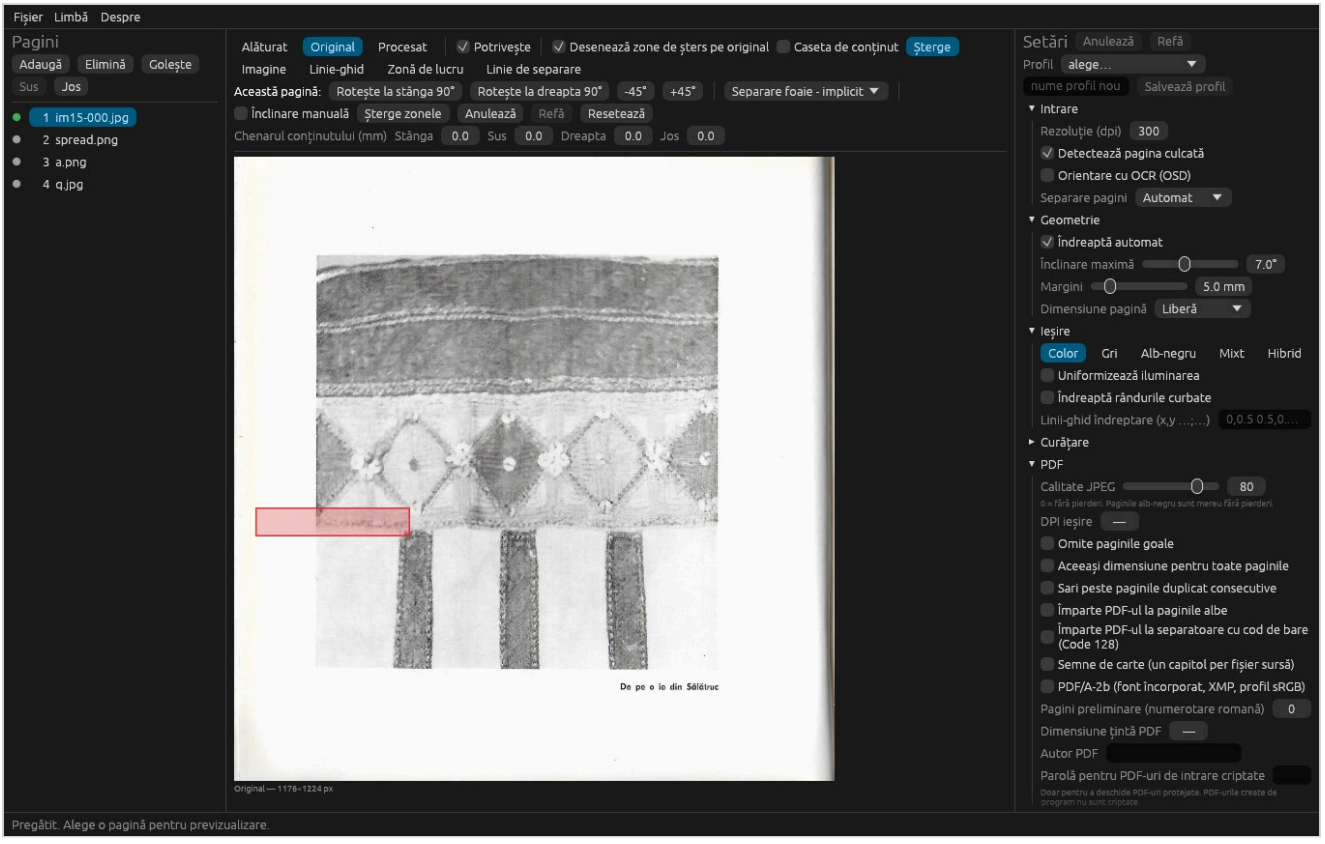
Upside-down pages. The “180°” button turns the chosen page (Ctrl+Z undoes it). In addition, on Latin-script text the program finds upside-down pages by itself, after deskewing: in text, ascenders and capitals stand out above the line more than descenders below it, and on an upside-down page the balance is reversed. It decides only when the page has at least 12 lines and at least 70 % of them “vote” the same way; otherwise it touches nothing. It does not work on other scripts and does not touch pages you turned by hand. Checked on synthetic pages and a few photographs; if a page stays upside down, press “180°”.

Zones, guides and the content box

Tick "Draw erase zones on the original"; five drawing types appear (Erase, Picture, Guide line, Work area, Cut line):

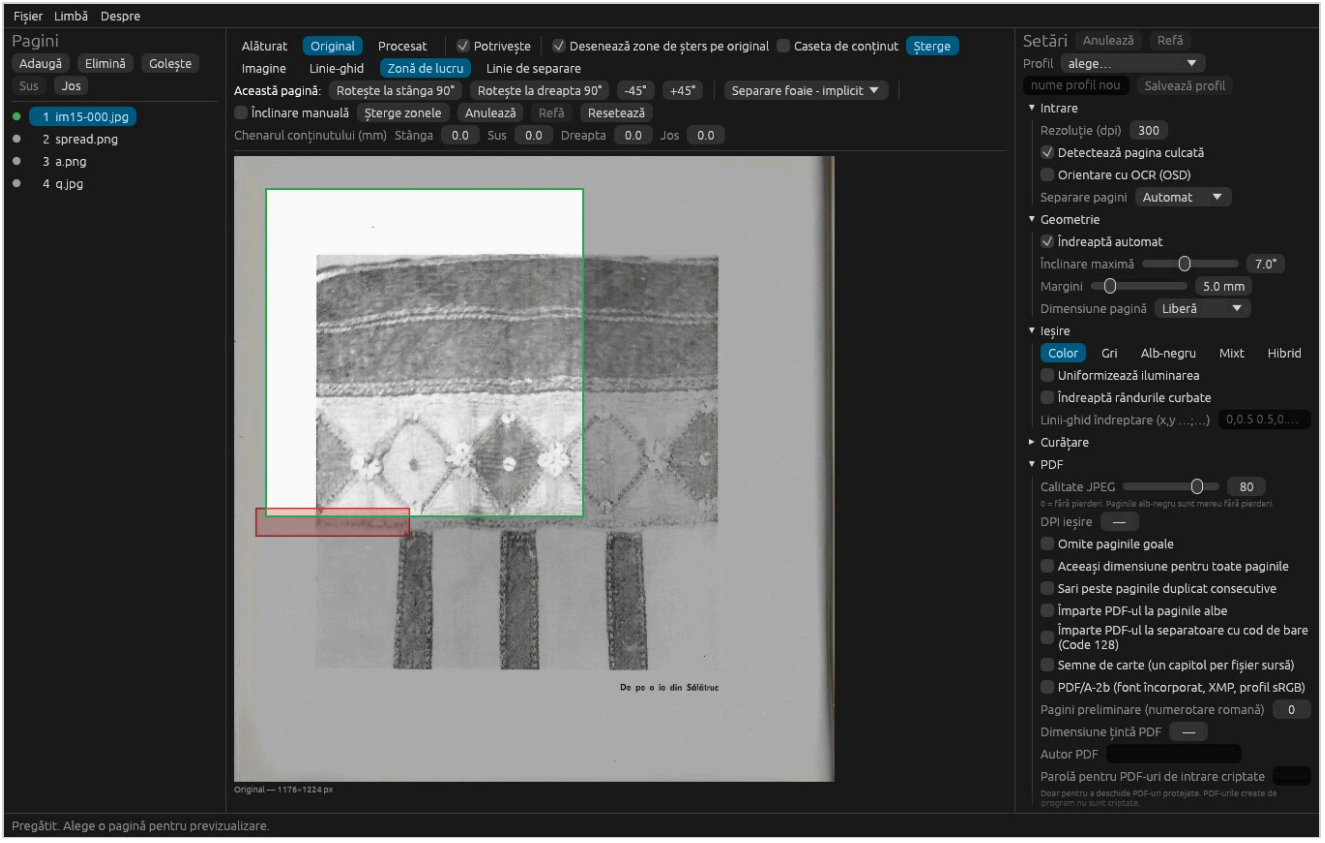


With "Draw erase zones on the original" ticked, the zone types appear: Picture, Guide line, Work area, Cut line (and Erase, the default type).

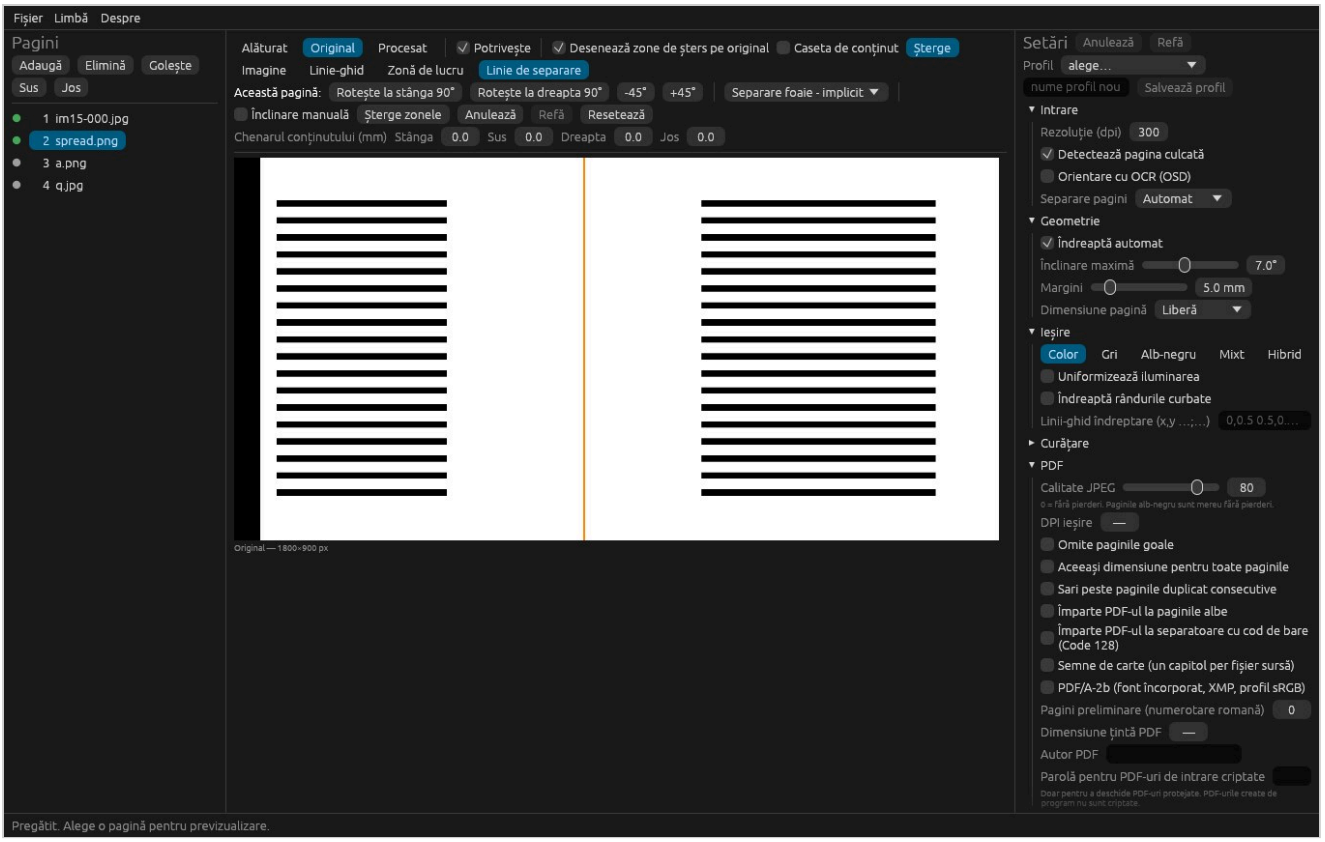


An erase zone (red) drawn with the mouse on the original, in the Original view.

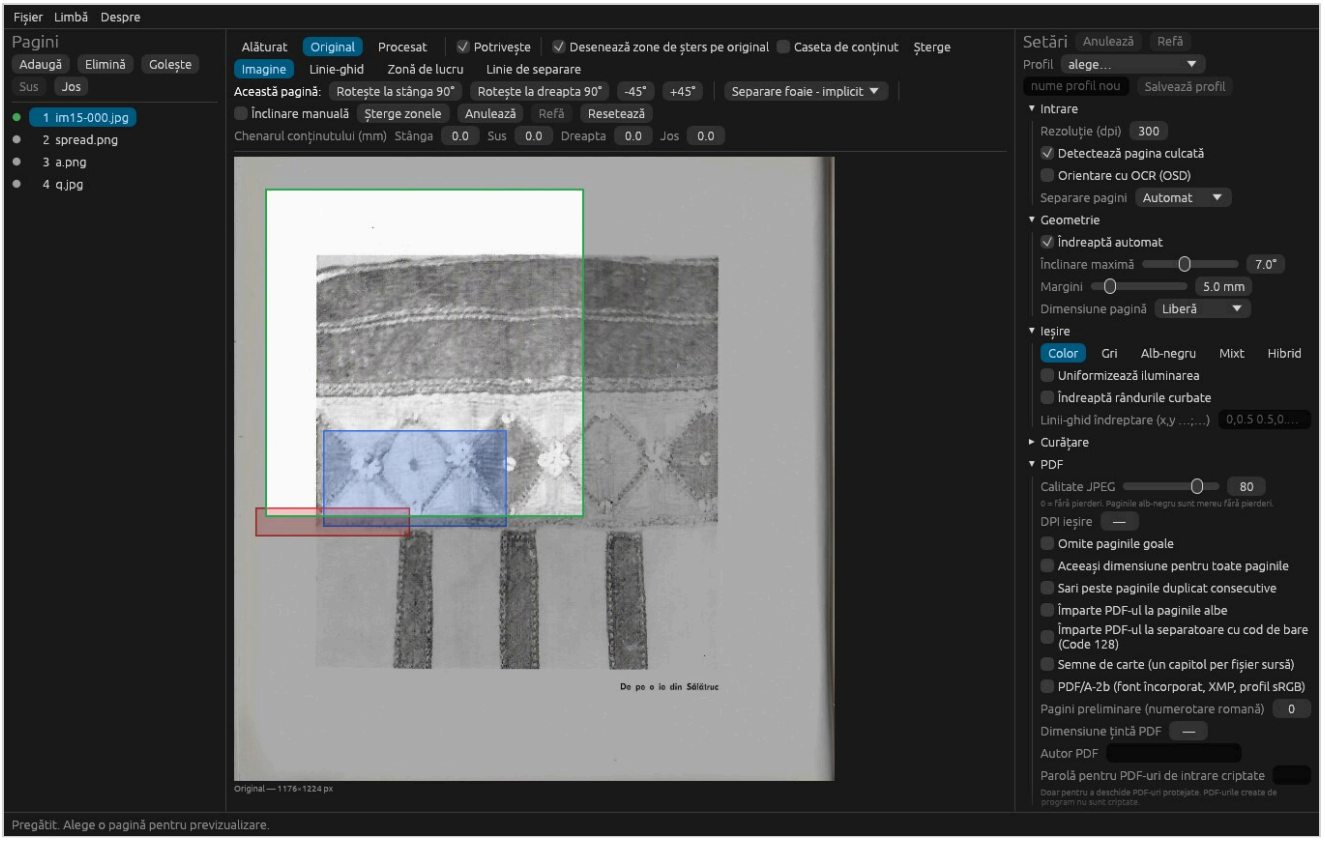
Type	How you use it	What it is for
Erase	Drag a rectangle on the original. Right-click on it deletes it. Drag a corner to resize it.	Stain, stamp, someone's note, the edge with a shadow
Picture	Same as "Erase".	Tells the program the zone is a picture, not text (it stays in colour in mixed mode)
Guide line	Drag freely along a curved line of text. Drag a node to move it; double-click on the line adds a node, double-click on a node deletes it; right-click on the line deletes it.	Straightens the page according to the lines you traced



The work area (green frame): everything outside it is removed, and the remaining area is seen in the processed page. It also has an erase zone (red).



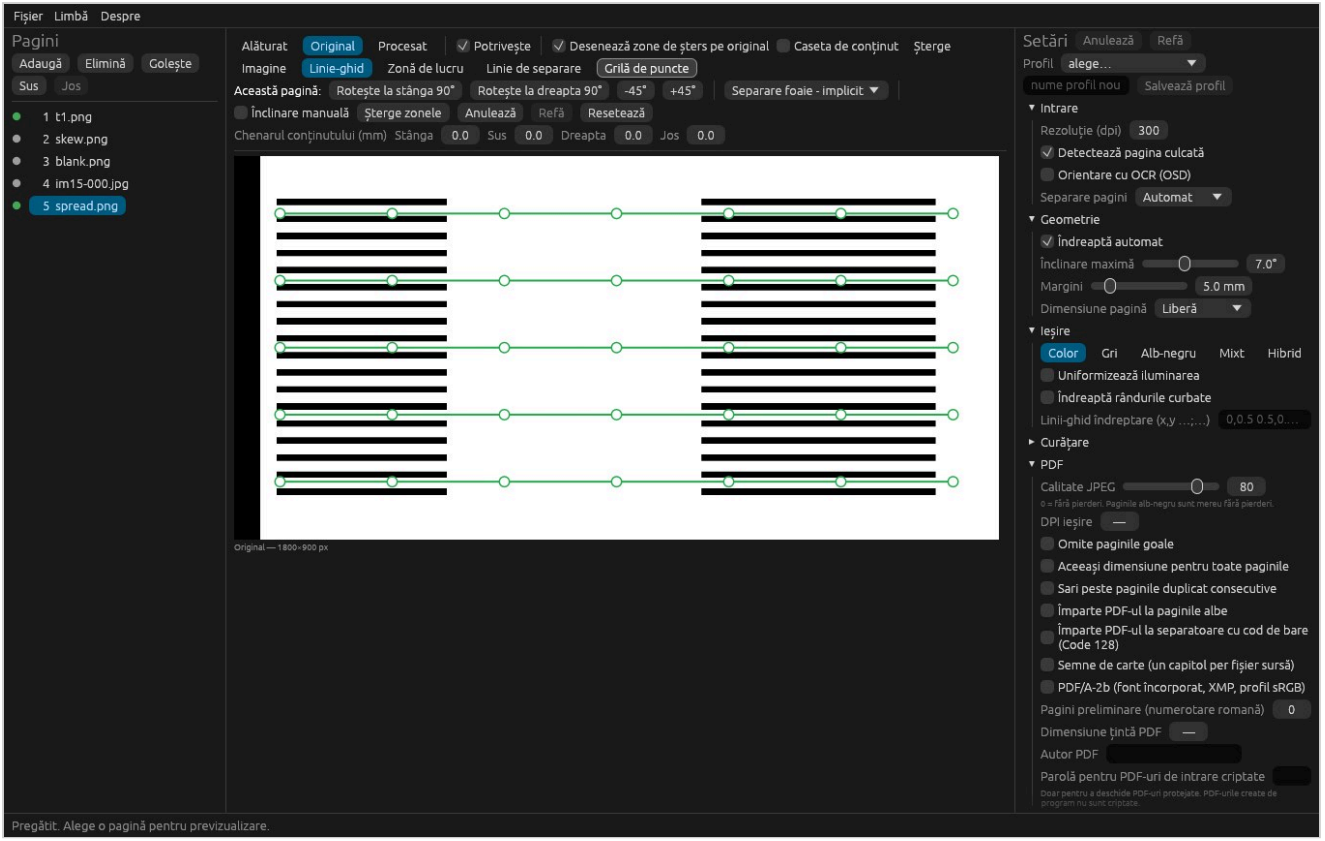
The cut line (orange) moved with the mouse on a double sheet. The example is a synthetic test sheet, with lines instead of text.



A picture zone (blue) drawn on the original. In Colour mode it changes nothing; the effect is seen in Mixed mode (next figure).



The same page and the same picture zone (blue): in Mixed mode the zone stays grey (left), in Black-and-white mode everything becomes black and white (right). The page appears rotated by 90° by the detection of a page lying on its side (see above).



"Grid of points" for Guide line: 5 lines of 7 points each, over the lines of text.

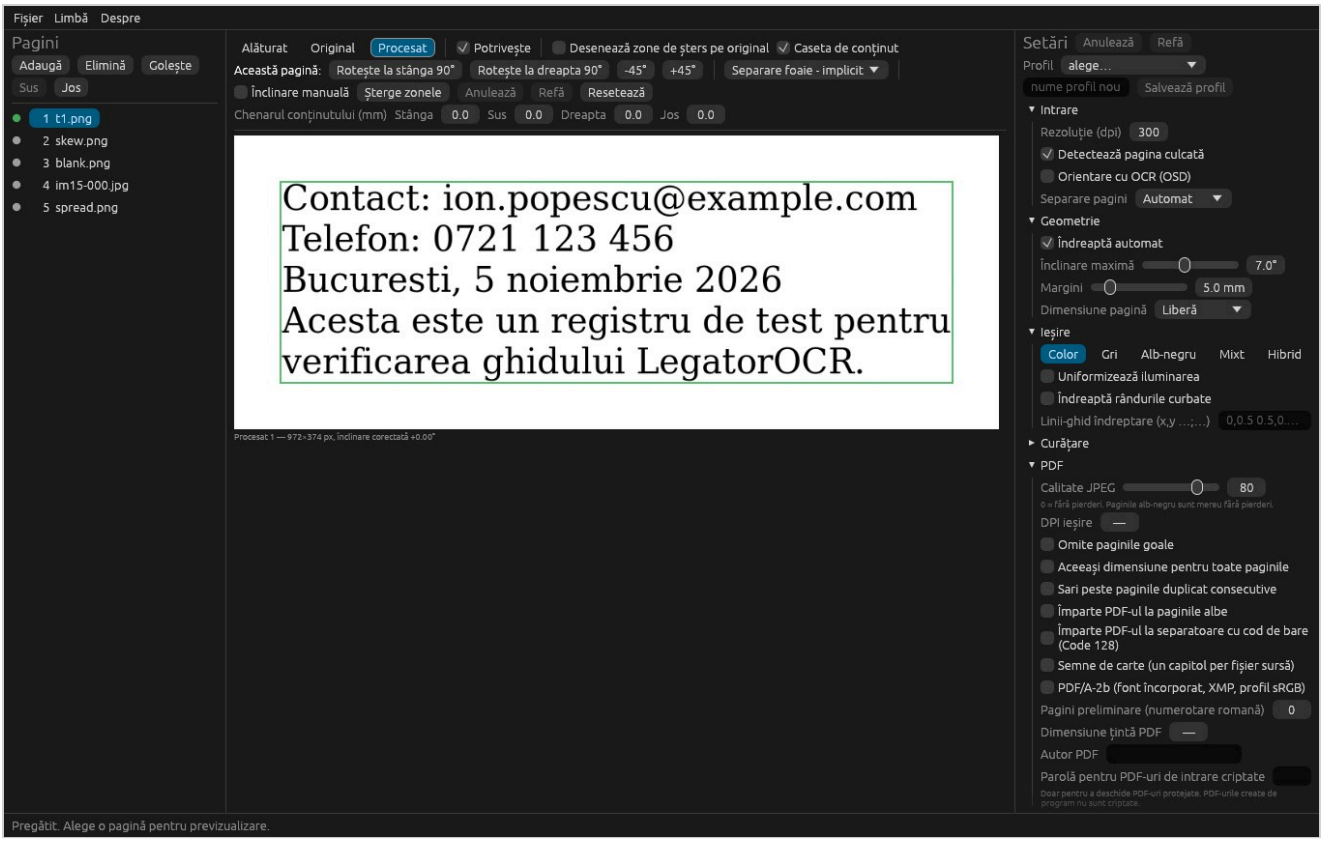
Grid of points (for *Guide line*) adds 5 lines of 7 points each, over which you drag the points onto the lines of text. A second click on the button does not add the same grid a second time. If you have already moved points, a new grid is added alongside the moved one.

Work area (since edition pre.18): you drag a rectangle on the original; everything outside it is painted with the paper colour before any measurement, so a dark edge or the neighbouring page no longer matters. Right-click on the original removes it. On the command line: `--work-area X,Y,L,H` (millimetres).

Cut line (since edition pre.18): by clicking or dragging on the original you move the orange vertical line; the double sheet is cut there, not at the middle or at the gutter. Since pre.21 it

works at any rotation; on sheets rotated by 90° or 270° the line cuts horizontally and the pages come out one below the other (the left page of the original first). Right-click removes it. On the command line: `--cut-at 0.45`.

The content box (the tick box of the same name) shows a green frame on the processed page. Drag the sides to widen or narrow the kept area. On a sheet cut into two pages, only the page you drag the box on changes.



The content box (green frame) on the processed page.

Bent pages, a slanted cut and settings per page

Three things for sheets photographed by hand:

- **Slanted cut line.** With "Draw erase zones on the original" ticked and the Cut line type, a "Line lean" slider (−20° to 20°) tilts the line; each page keeps its side of the slanted line and the other side is painted with the paper colour. Right-click the line to remove it and reset the lean. On the command line: `--cut-angle DEGREES`.
- **Keystone correction.** Two sliders in the "This page" bar ("Keystone top–bottom", "Keystone left–right", ±30 %) straighten a sheet photographed at an angle before the cut. The **Auto keystone** button (after the pages are split) measures each page — the tilt of the text lines and the width of the text at top and bottom — and keeps a correction for that page only. It is careful on purpose: when there are too few lines or the measurements contradict each other it leaves the page as it is. Check the result, with Undo at hand. The −45°/+45° buttons stay. On the command line: `--keystone-v` and `--keystone-h` (percent).
- **Settings per page.** "Settings for: All pages / This page only", above Output and Clean-up, applies the output mode, threshold, despeckle and the other clean-up settings to a single page of a cut sheet; the green content box changes only the page it is dragged on. In a project: `page1.KEY = VALUE` / `page2.KEY = VALUE` under [sheet]. A click on a processed page

selects it (blue frame): the output mode (gray, black-and-white, mixed), clean-up, keystone, the green box and the OCR pencils then work on that page alone. OCR zones you draw stay with that page (`page1.page_ocr_zones`, `page_ocr_ignore`).

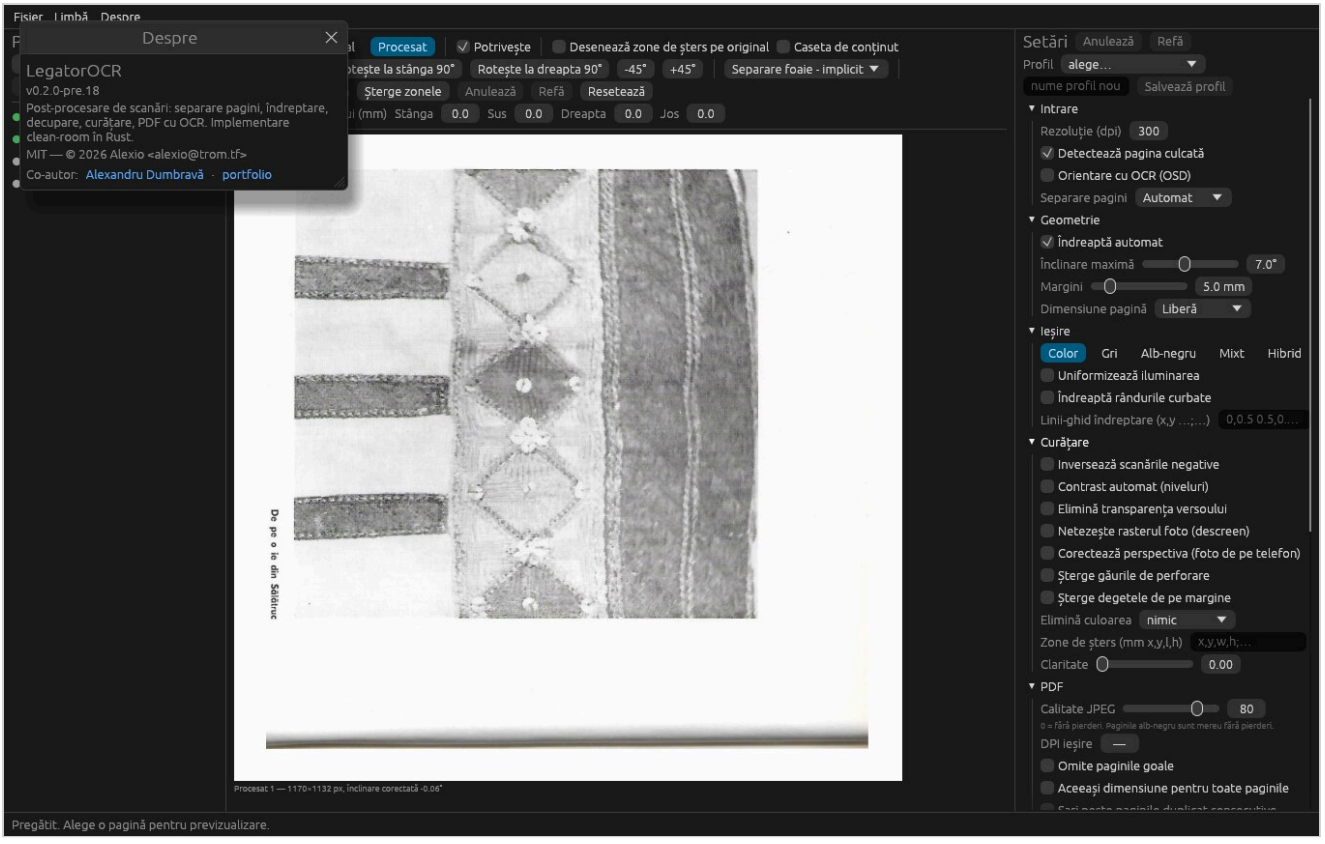
The bar below the preview holds the content box, the OCR pencils and **Auto OCR pencil** (separately for the left and the right page): it puts the OCR zone on the content of that page.

The manual tilt works on the selected page (left or right) of a cut sheet: each page can have its own angle (`page1.page_skew`, `page2.page_skew`).

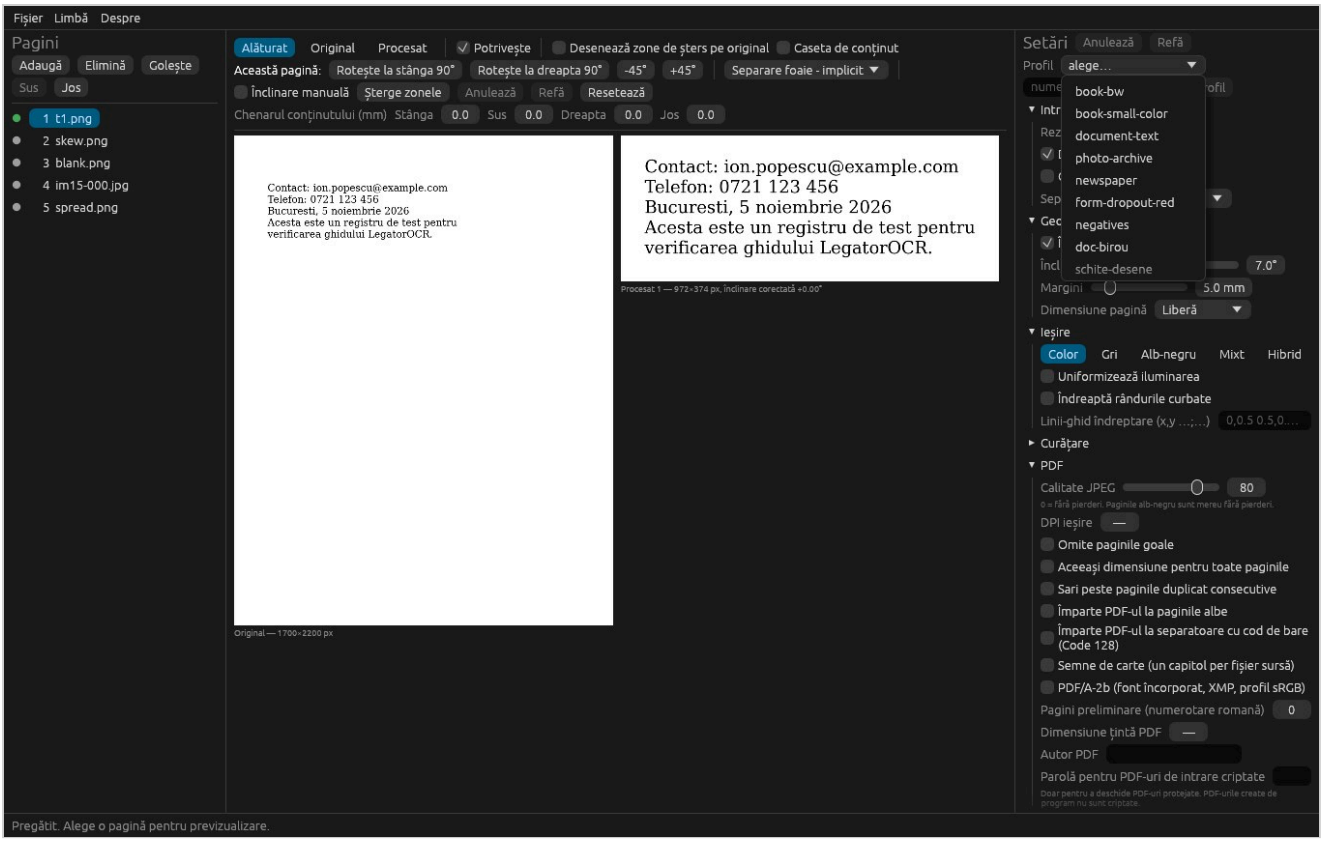
Page sizes now include A2–A6, B3–B6, Letter, Legal and Tabloid (`--page-size` takes the same names). OCR pencils on processed pages: green (OCR zone), red (ignored area), violet (table, read as one block row by row; `--ocr-zone X,Y,W,H#table`).

The settings on the right

They are grouped into sections: Input, Geometry, Output, Cleaning, PDF, OCR, plus the profiles. Section 6 explains what each one does.



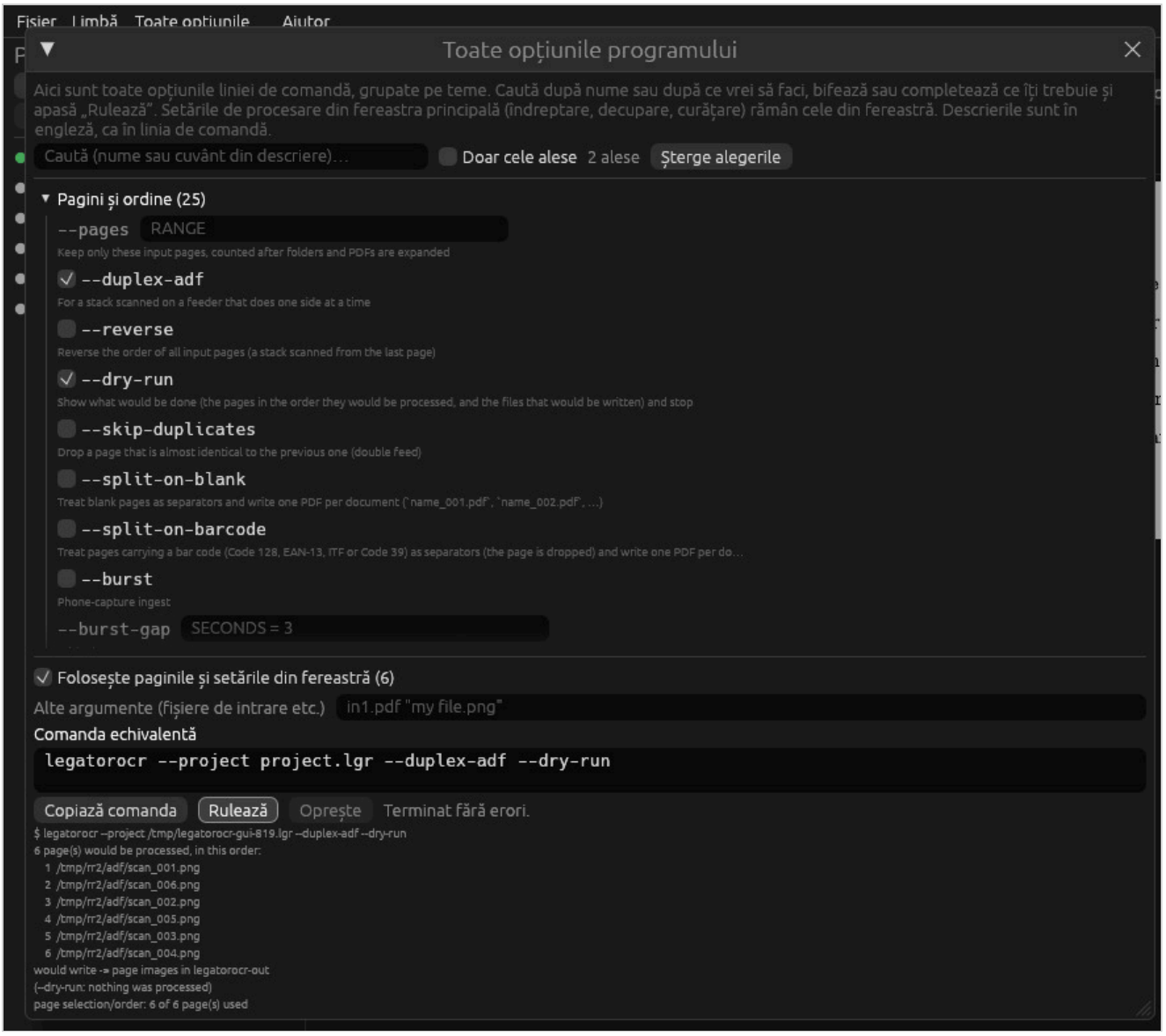
The Cleaning section of the settings, expanded: inversion of negative scans, automatic contrast, perspective, removal of punch holes, erase zones as numbers.



The "Profile" list in the settings (the built-in profiles; yours appear in the same place after "Save profile").

All options and the user guide

The **All options…** button in the top bar opens a window with **all** the command-line options (over 200), grouped by topic: pages and order; straightening, cropping and cleaning; outputs (PDF, images, documents); OCR and text; archiving, signing and verification; search and comparison; batches, system and modules. The list is read from the command-line program shipped with the window itself, so it always matches the edition you have.

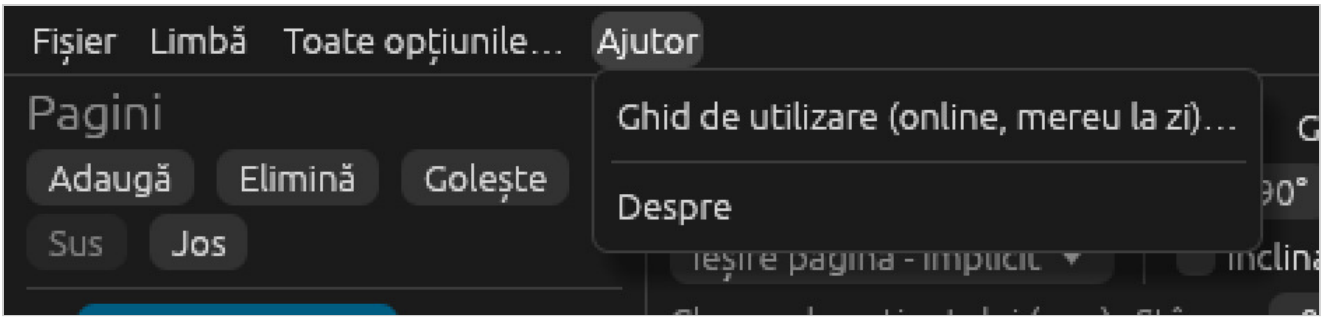


The "All options" window, with the "Pages and order" group open, `--duplex-adf` and `--dry-run` ticked and the result of the run: the order in which the six pages of the stack would be processed (test pages made by me).

- **Search** by name (`pdf`, `dry`) or by a word from the description; the groups with results open by themselves. "Only the chosen ones" shows what you have ticked or filled in.
- Empty boxes are not sent: an option with a value enters the command only when you type something (or choose from a list). The **Choose…** button opens the file or folder dialog. For options that can be repeated (for example `--erase`) you write one value per line.
- **Use the pages and settings from the window** (ticked by default): the command receives the loaded pages, with the adjustments of each, through a temporary project. The processing settings of the main window (straightening, cropping, cleaning) take priority; here you fill in the outputs and the special functions. Untick it for commands without pages (`--doctor`, `--verify-sums`) or write the input files in "Other arguments".
- **The equivalent command** is updated as you choose; you can copy it and use it in a terminal or in scripts. **Run** executes it and shows the output below; **Stop** interrupts it. Relative paths are relative to your home folder.

- The option descriptions are in English, as on the command line; the rest of the window is in the chosen language.

The **Help** menu has the link to this guide: *User guide (online, always up to date)* opens the latest version published on legatorocr.chiuta.ro, and *Guide of this edition* (it appears when the PDF file is next to the program, as in the published archives) opens the copy shipped with the program. The guide is rebuilt for each edition, and the publication of an edition is stopped if the guide does not mention its version.



Checked: the window, the search, the ticking, the running of a command with the pages from the window (the screenshot above) and the opening of the Help menu, on Linux, on a virtual screen. Not checked: Windows, macOS and arm64; the actual opening of the guide in a browser (I only checked the menu); the "Choose…" dialogs; the translations of the window's new texts (written automatically).

Collections

A collection gathers newspapers, magazines, books and the like in one `.lgrcol` file (JSON). Keep it in the folder with the scans: paths inside the file are written relative to it when the target lies under a folder they share, so the folder can be moved or copied. Each item has a kind, title, issue or volume, date or year, languages, sources (images, a folder of images, a PDF, a saved `.lgr` project), the PDF made from it, a profile, a status (to do, processed, OCR done, checked), page count, tags and notes.

```
legatorocr collection new Archive/ --name "Old papers"
legatorocr collection add Archive/old_papers.lgrcol --folder Archive/scans
legatorocr collection list Archive/old_papers.lgrcol
legatorocr collection run Archive/old_papers.lgrcol --out Archive/pdf
legatorocr collection search Archive/old_papers.lgrcol "Scînteia"
legatorocr collection reindex Archive/old_papers.lgrcol
legatorocr collection set Archive/old_papers.lgrcol i1 --status checked --tags "1977, youth"
legatorocr collection remove Archive/old_papers.lgrcol i2
```

- `add --folder`: each PDF, `.lgr` project and subfolder of images becomes an item. Title, kind, date and issue are guessed from the name: `scinteia_tineretului_1977_09_8801.pdf` gives a newspaper, "Scinteia Tineretului", 1977-09, no. 8801. It is only a guess (file names carry no diacritics; the kind rests on a few key words and on the numbering: a full date or an issue number of 500 or more is taken for a newspaper); correct it in the window. For `name_year_month_number` a number of three or more digits is the issue (`..._1977_09_8801`); a number of one or two digits right after the month is a day only if it is a real day of that month and, when it is the last part, the name does not say magazine or book (`gazeta_1914_05_12` is the 12th of May, `revista_sinteza_1985_03_12` is issue 12, `..._1977_09_12_8801` is the 12th, issue 8801).

- `run` processes the items whose status is "to do", each with its own profile (else the collection's, else the engine defaults), through the same code as the command line and the window (`run_export`). An item that fails (missing source, unknown profile, unreadable page) is marked with the reason and the others go on; the collection file and the index are saved after every item, so a cancel or a crash loses nothing that was done. PDFs go next to the collection file, or to `--out`. With OCR (on by default; `--no-ocr` turns it off) the recognised text of every page goes to `NAME.lgrindex`; the OCR language comes from the item's languages, else the profile's `ocr_langs`, else Romanian. An item that failed is not tried again unless `--retry-failed` or `--item ID` is given.

- `search` looks in that index. Case, diacritics and the old Romanian spelling do not matter (`Scînteia` finds `Scânteia`, `sînt` finds `sunt`). It prints item, page and a snippet: pages with the words next to each other first, then (marked) pages that have all the words apart.

- `reindex` (alias `index`) reads the text layer of PDFs into the same index **without processing anything again**: for an item whose source is one PDF that already has text (searchable PDFs from another program, or made by LegatorOCR) and for items processed earlier. It reads the PDF made by `run` if there is one, else the item's single PDF source; the item's status and files are not changed. A scan without text is reported (`no text layer`) and not indexed (process it with OCR); an item with several sources, or only images, has no PDF to read. `--item ID` restricts it. The text is taken as the PDF has it (fonts with a ToUnicode table, as written by LegatorOCR, Tesseract, ABBYY, Word …); a PDF whose text cannot be decoded gives nothing or garbage.

- `set FILE ITEM` changes `--title --issue --date --kind --profile --status --tags --notes` (only the ones given; an empty text clears a field; a bad value changes nothing; `--status todo` makes the next `run` process the item again). `remove FILE ITEM...` takes items out of the collection and of the index; their PDFs and source files are not touched.

- The **Collections…** window (File menu, or Ctrl+K): a table with kind letter, title, issue/date, pages and a status chip (colour and text), with filter and sort; the details on the right; Add files, Add folder, Process selected, Process all pending (with progress and Cancel), Index text / Index all (the `reindex` above), Open PDF, Show folder; a search box over the whole collection with clickable results. Double-clicking a result opens the PDF at that page when the computer's default PDF program is one known to honour a page number (Linux: Okular, Evince, Zathura, MuPDF, Firefox, Chrome, Chromium, Edge; Windows: SumatraPDF, Adobe Acrobat/Reader, Chrome, Edge, Firefox; found by asking the system which program opens PDFs); otherwise the PDF opens normally and the page number is written in the status line. Processing from the window uses the Tesseract set in the main window's settings (an item's profile may still name another). Its texts exist in the 8 languages of the window.

Tested: unit tests (name guess including the year-month-number patterns, relative paths, damaged files, search with diacritics, a batch that goes on after a failure and after a panic, cancel, the index file with and without its cache, index files of the old format, a truncated or damaged index, reindexing a PDF with and without text, `set` and `remove`, the viewer commands of each platform), tests of the real binary, and one run with real Tesseract on two synthetic pages, also after moving the folder. **Measured** on a synthetic index of 5000 items × 20 pages (100,000 pages of 350 words, text file 357 MB; release build on a shared 2-core machine, so orders of magnitude only): opening takes 62 ms with the word-list cache (38 MB) and 12 s without it (first opening, or the file was changed by another program); 82 MB of memory after opening (the earlier index, kept whole in memory: 34 s and about 480 MB); a search for one word 40 to 130 ms, for two very common words that are never side by side 3 s.

Limits: the window was written and compiled but nobody has looked at it on a screen or clicked through it, so its new buttons, the status line and the opening at a page are untried; the viewer commands are unit-tested only as text, never run against Okular, Acrobat or the others; on macOS and with any other PDF program the page is not passed, only shown in the status line; the page of a hit is the page of the PDF that was read (for a processed item, the PDF made by `run`), so after replacing that PDF by one with other pages run `reindex`; `reindex` does not run OCR and cannot read text that a PDF does not decode; the index is rewritten whole at each save (about 2 s for 357 MB), so changing a very large one is slower than a small one; two programs writing the same files at once: the last one wins; the old-spelling rule is the one of `--ocr-modern-spelling` and can join different words (`mână` / `mină`).

5. The command line: recipes for frequent tasks

General syntax: `legatorocr INPUTS... [options]`. The inputs can be images (PNG, JPEG, TIFF, BMP), scanned PDFs or a folder. `legatorocr --help` lists all the options; `--list-profiles` shows the profiles. The options in the recipes are checked automatically before each edition, so they really exist; the recipes themselves have not all been tried on real material.

Scanned book → PDF with searchable text

```
legatorocr scans/*.png --profile book-bw --ocr --lang ron --pdf carte.pdf --pdfa --txt
carte.txt -o pages
```

`--pdfa` writes PDF/A-2b for archiving, and `--txt` also saves the text. `-o pages` is the folder for the processed images.

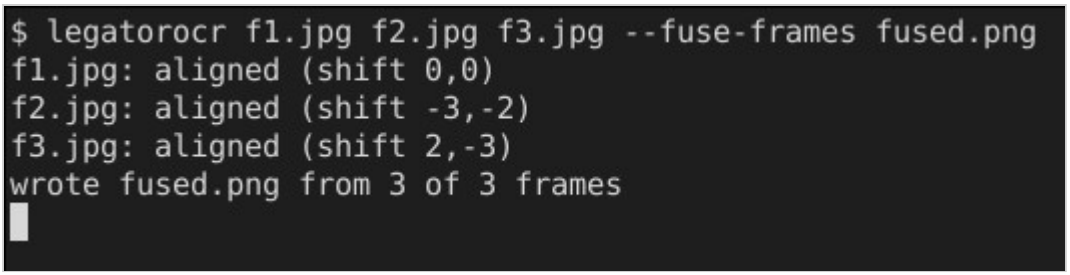
```
$ legatorocr scans/*.png --profile book-bw --ocr --lang ron --pdf carte.pdf --pdfa --txt carte.txt -o pagesT
OCR engine: tesseract 5.3.4, language: ron
a_L: 304x164 px @ 300 dpi, skew +0.00°
a_R: 284x254 px @ 300 dpi, skew +0.00°
p15b: 1162x1232 px @ 300 dpi, skew +0.00°
spread_L: 524x818 px @ 300 dpi, skew +0.00°
spread_R: 674x818 px @ 300 dpi, skew +0.00°
text: 972x374 px @ 300 dpi, skew +0.00°
wrote 6 page image(s) to pagesT
wrote carte.pdf (6 page(s), 27 OCR words)
wrote carte.txt
```

The recipe on 4 test files: 6 pages in carte.pdf. With --pdfa the program writes PDF/A-2b.

Phone photos → clean pages

```
legatorocr photos/*.jpg --profile phone-photos --pdf pagini.pdf -o pages
```

The profile evens out the lighting, corrects perspective, wipes the background around the page (`--trim-background`), cuts open books in the middle, widens the skew range and removes fingers without touching the text. A page that comes out without text is read a second time on its content only. For uniform A4 pages, with enlarged text, use `--profile phone-photos-a4`. Several photos of the same page can first be combined with `--fuse-frames` (it helps with strong noise; with moderate noise it does not). The option takes an output file: legatorocr photo1.jpg photo2.jpg photo3.jpg --fuse-frames page.png, then you process page.png as usual; the first photo is the reference.



--fuse-frames with three photos of the same page: it aligns them and writes fused.png from 3 of 3 frames.

Archive copy with proof of integrity

```
legatorocr scans/*.png --profile registre-manuscrise --ocr --lang ron --pdf registru.pdf --
pdfa --checksums --archive-package registru-arhiva -o pages
legatorocr --verify-sums registru-arhiva/SHA256SUMS.txt
```

The package keeps the originals, the project, the OCR words with confidence, the outputs and a `RUN.txt` that says how to resume the work. For copies on other disks: `--backup-from` and `--backup-to`.

```
$ rm -rf registru-arhiva2; legatorocr scans/text.png --profile registre-manuscrise --ocr --lang ron --pdf registru2.pdf
--pdfa --checksums --archive-package registru-arhiva2 -o pagesT2; legatorocr --verify-sums registru-arhiva2/SHA256SUMS.t
xt
OCR engine: tesseract 5.3.4, language: ron
text: 972x374 px @ 300 dpi, skew +0.00°
wrote 1 page image(s) to pagesT2
wrote registru2.pdf (1 page(s), 20 OCR words)
wrote registru-arhiva2/derived/ocr.json
archive package: registru-arhiva2 (6 files, 1 originals)
wrote ./SHA256SUMS.txt (2 file(s) added or updated)
5 ok, 0 mismatched, 0 missing
```

The archive package created and verified: "5 ok, 0 mismatched, 0 missing".

Hiding names and numbers before sharing

```
legatorocr scan.png --ocr --lang ron --redact email,phone-ro --format none --pdf redactat.pdf
```

The words found are blacked out in the image and removed from the text. **Check the result with your own eyes:** a word that OCR did not recognise is not hidden.

```
$ legatorocr scans/text.png --ocr --lang ron --redact email,phone-ro --format none --pdf redactat.pdf; pdftotext redacta
t.pdf - | head -4
OCR engine: tesseract 5.3.4, language: ron
text: 972x374 px @ 300 dpi, skew +0.00°
redaction: preset email: 0 covered; preset phone-ro: 1 covered;
warning: preset email matched nothing in 1 page(s). OCR may have misread the value (an "@" is often read as "G" or "(a")
: look at the pages yourself before you hand the files over.
warning: 1 word(s) left in the text still look like personal data (page 1: address-like word). They were NOT covered; ch
eck them by eye.
wrote redactat.pdf (1 page(s), 17 OCR words)
Contact: ion.popescuGexample.com

Telefon:
Bucuresti, 5 noiembrie 2026
```

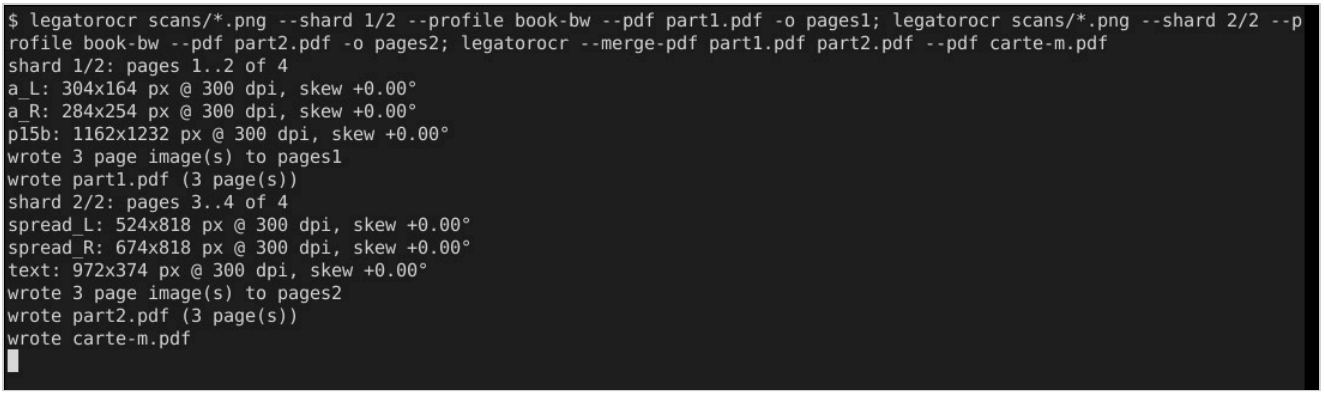
Real result: the phone number was hidden, but the e-mail address was NOT, because OCR read "@" as "G" (ion.popescuGexample.com) and the e-mail pattern did not match. This is why you must check the result with your own eyes. Since pre.20 the program writes at the end how many

words each pattern covered and warns when a pattern found nothing or when words that look like addresses or long numbers remain in the text, exactly as in the screenshot above.

A large batch on several computers

```
legatorocr scans/*.png --shard 1/3 --profile book-bw --pdf part1.pdf -o pages1
legatorocr --merge-pdf part1.pdf part2.pdf part3.pdf --pdf carte.pdf
```

Run the same list on each machine, with a different `--shard`. For low memory: `--max-memory 500`. For unknown or hostile files: `--isolate-decoders`.



The batch split in two (--shard 1/2 and 2/2) and then joined with --merge-pdf.

Signing a PDF

```
LEGATOROCR_P12_PASS=... legatorocr --sign-pdf registru.pdf --sign-p12 certificat.p12 --sign-
name "Name"
legatorocr --verify-pdf-signatures registru.signed.pdf --sign-ca ca.pem
```

The legal level of the signature is that of your certificate. The certificate password is given through the environment variable, not on the command line. The time stamp (`--sign-timestamp`) was checked only with a local authority over HTTP, not with a real one over HTTPS.



Signing and verification with a test certificate created by me (not a real certificate). "time stamp inside: none": I did not request a time stamp.

A stack scanned on a feeder, page order and range

Many scanners with a feeder pass the stack once for each side: first all the fronts (1, 3, 5…), then all the backs, in reverse order (6, 4, 2). The files come out in scanning order, not in the order of the book. `--duplex-adf` puts them in reading order: front 1, back 1, front 2, back 2… The number of pages must be even (otherwise the program stops with a message).

```
$ ls adf; legatorocr adf/*.png --duplex-adf --dry-run
scan_001.png  scan_002.png  scan_003.png  scan_004.png  scan_005.png  scan_006.png
page selection/order: 6 of 6 page(s) used
6 page(s) would be processed, in this order:
    1  adf/scan_001.png
    2  adf/scan_006.png
    3  adf/scan_002.png
    4  adf/scan_005.png
    5  adf/scan_003.png
    6  adf/scan_004.png
would write -> page images in legatorocr-out
(--dry-run: nothing was processed)
```

Six files in scanning order (scan_001 … scan_006). `--dry-run` shows the order in which they would be processed and which files would be written, then stops without processing anything: scan_006 ends up in position 2, scan_005 in position 4.

```
$ legatorocr adf/*.png --duplex-adf --engine native --ocr --lang ron --pdf carte.pdf --txt carte.txt 2>&1 | tai
$ grep -h Pagina carte.txt | tr -d '\f' | tr '\n' ' '; echo
wrote carte.pdf (6 page(s), 240 OCR words)
wrote carte.txt
Pagina 1 Pagina 2 Pagina 3 Pagina 4 Pagina 5 Pagina 6
```

The same command without `--dry-run`, with native OCR: the text comes out in the order "Pagina 1 … Pagina 6" (test pages made by me, not a real book).

Other ordering and selection options (they can be combined; the range applies to the numbering of the input files, starting from 1):

- `--pages 1-10,15,20-`: keeps only the pages indicated; `20-` means "from 20 to the end". It also works on pages taken from PDFs or from multi-page TIFF sheets.
- `--reverse`: reverses the order (a stack scanned from the last page).
- `--dry-run`: checks a large batch before the long run.

Checked: the order and the range with automated tests and with six test pages; I did not try a real scanner with a feeder. The options can also be chosen from the window, under "All options…" (section 4).

Clean text: your own corrections, running heads and cedilla

For the `.txt` and `.docx` files (the text layer in the PDF, hOCR and JSON keep the words as read, unchanged):

- `--clean-text`: ș and ț with a comma below, words broken at the end of a line joined (with the hyphen kept when the two halves are words from the list and simple joining is not), typographic quotation marks, spaces next to punctuation. `--cedilla` writes ş and ţ instead, for programs and fonts that do not know the comma letters.
- `--find-replace FILE`: your own corrections, one per line, `wrong => right`. Lines that start with `#` are comments. Only whole words match (`Ion => Ioan` does not touch "Ionescu") and they are applied in the order of the file, after `--clean-text`. Useful for proper names, abbreviations or a stamp that the engine always misreads in the same way.
- `--drop-running-heads`: leaves out the printed running heads, footers and page numbers: the first or last line of a page that repeats (digits do not count) on at least half of the pages; at least four pages are needed. The lines removed are listed on stderr, so that you can see what went.

```
$ cat reguli.txt
O="--engine native --ocr --lang ron"
legatorocr rev/p01.png $O --txt f0.txt 2>/dev/null
legatorocr rev/p01.png $O --txt f1.txt --find-replace reguli.txt 2>/dev/null
echo "fără regulă:"; grep -h "secretarul\|ha\." f0.txt
echo "cu regulă:"; grep -h "secretarul\|ha\.\|hectare" f1.txt
# corecturi proprii
Ionescu => Ionescu-Vale
ha => hectare
Volumului => Volumul
fără regulă:
în anul 1951 de către secretarul Ionescu.
și suprafața înscrisă în ha. Corecturile se
cu regulă:
în anul 1951 de către secretarul Ionescu-Vale.
și suprafața înscrisă în hectare. Corecturile se
```

The rules file (first block) and the same two lines read from the test page without and with `--find-replace`: "Ionescu" became "Ionescu-Vale" and "ha" became "hectare".

```
$ legatorocr rev/*.png --engine native --ocr --lang ron --clean-text --drop-running-heads --txt rev.txt; head -
OCR engine: LegatorOCR native engine 0.2.0-pre.26, language: ron
p02: 996x904 px @ 300 dpi, skew +0.00°
p01: 992x904 px @ 300 dpi, skew +0.00°
p04: 998x904 px @ 300 dpi, skew +0.00°
p03: 998x904 px @ 300 dpi, skew +0.00°
p05: 998x904 px @ 300 dpi, skew +0.00°
p06: 998x904 px @ 300 dpi, skew +0.00°
wrote 6 page image(s) to legatorocr-out
6 running head/foot/page-number lines left out of the text (e.g. "Anuarul Văii Mari 1", "Anuarul Văii Mari 2",
Văii Mari 3")
wrote rev.txt
Pagina 1
Registrul comunei Valea Mare a fost rescris
în anul 1951 de către secretarul Ionescu.
Fiecare pagină păstrează numele proprietarilor
```

On six test pages with the running head "Anuarul Văii Mari" and a number, the program removed six running-head lines; the text starts directly with "Pagina 1". Note: the rule removes any first/last line that repeats, so in a book where the last line of each page is similar (for example "Shelf position: 14") those lines would go too; this is why they are listed. The function was checked with tests and on a few pages, not on a large set of books.

Other recipes

The complete recipes are in `docs/tasks/` in the repository, in English and Romanian (document verification with `--verify-document` and `--mrz`, archive copy, large batch).

6. Important settings and when to use them

Profiles

```
$ legatorocr --list-profiles | cut -c1-100
book-bw              Text books: auto threshold, black & white, punch holes removed
book-small-color     Colour books: sharp text over a light background, smallest files
document-text        Office documents: sharp text, pictures kept in colour
photo-archive        Archive quality colour, light sharpening
newspaper            Newsprint: Wolf threshold, stronger speck removal, flattened curl
form-dropout-red     Forms with red rules or stamps: drop red, erase ruled lines
negatives            Negative scans: invert and stretch contrast
doc-birou            Office documents and invoices (scan at 300 dpi): black & white, searchable PDF
schite-desene        Architectural / pencil sketches (scan at 400 dpi): grey, even lighting, light c
registre-manuscrise  Old registers and manuscripts (scan at 300-600 dpi): colour kept faithfully, hi
flux-1-optim         Flux 1 (optim, scan at 300 dpi): sharp 1-bit text, pictures in colour, margins
flux-2-draft         Flux 2 (ciornă / draft): quick grey pass, light compression - for checking and
flux-3-volum         Flux 3 (volum mare / high volume): 1-bit text over a very light background, sma
diploma              Diplomas and certificates (scan at 300-400 dpi): colour seals, stamps and signa
registru-stare-civila Civil-register pages with handwriting and stamps (scan at 400 dpi): colour kep
harta                Maps (scan at 400 dpi or more): colour kept faithfully, no thresholding, no des
partitura            Music scores (scan at 600 dpi): fine 1-bit with a local threshold so thin staff
phone-photos         Pages photographed by phone: level lighting, wider skew range, fingers removed
smokeprof            user profile
```

legatorocr --list-profiles. The last line (smokeprof) is a test profile of mine, not part of the program.

A profile is a ready-made set of settings for a type of material. It is chosen from the "Profile" section (interface) or with `--profile NAME` (command line). You can have your own profiles: you configure the settings, type a name and press "Save profile".

Profile	What for
`book-bw`	books with text, black and white, punch holes removed
`book-small-color`	colour books, clear text on a light background, small files
`document-text`	office documents, images kept in colour
`photo-archive`	photographs, archival quality
`newspaper`	newspapers: Wolf threshold, stains removed more strongly
`form-dropout-red`	forms with red lines or stamps
`negatives`	negative scans
`phone-photos`	phone photos: perspective corrected, background around the page wiped, open books cut in the middle, fingers removed
`phone-photos-a4`	like `phone-photos`, but each page on an A4 sheet, with the text enlarged to fill the sheet (also pages with few lines)
`registre-manuscrise`	old registers and manuscripts (300–600 dpi), colours kept
`schite-desene`	sketches and pencil drawings (400 dpi)
`doc-birou`, `flux-1-optim`, `flux-2-draft`, `flux-3-volum`	flows for office documents, from maximum quality to high volume
`diploma`, `registru-stare-civila`, `harta`, `partitura`	**not verified on real samples**

The complete list is given by `--list-profiles`.

The settings that matter most

Setting	What it does	When you change it
Resolution (dpi)	what resolution the pages were scanned at	when the image has no dpi in its header; for text 300
Auto-deskew / maximum skew	the limit of the automatically corrected angle	when a page that comes out "borderline" is in fact crooked
Margins, page size, same size	margins in mm; free page size, custom or identical for the whole book	for books with mixed formats
Output mode	colour, grey, black and white, mixed (1-bit text + colour images), hybrid (+ low-resolution background)	black and white for pure text; mixed for text with pictures
Threshold method	Sauvola (default), Wolf, Niblack, NICK, Bradley, Otsu or automatic	when the text comes out faded or full of stains; `auto` tries several on each page
Sensitivity	the sensitivity of the Sauvola method for black-and-white output	default 0.3; change it in small steps and compare the result on the same page
Picture sensitivity	how easily an area is taken to be a picture	in mixed/hybrid mode
Remove small specks	the strength of speck cleaning (0.25–4; 1 = default)	lower it if dots or diacritics disappear; raise it if specks remain
Dewarp (curved lines)	straightens the arched lines near the spine	for thick books; skipped if no clear curvature is seen
Even out lighting	evens out the background	for photos with shadow or uneven lighting
JPEG quality	0 = lossless; black-and-white pages are always lossless	for smaller files (suggestion: try 80–90 and compare size and appearance)
Output DPI	shrinks the output image to this dpi ("—" = do not shrink)	for smaller PDFs; pages under 250 dpi are automatically enlarged before OCR, so too big a reduction does not help recognition
PDF/A-2b	PDF for archiving, with font and colour profile included	for deposit in archives; it has not been checked with veraPDF on the program's outputs

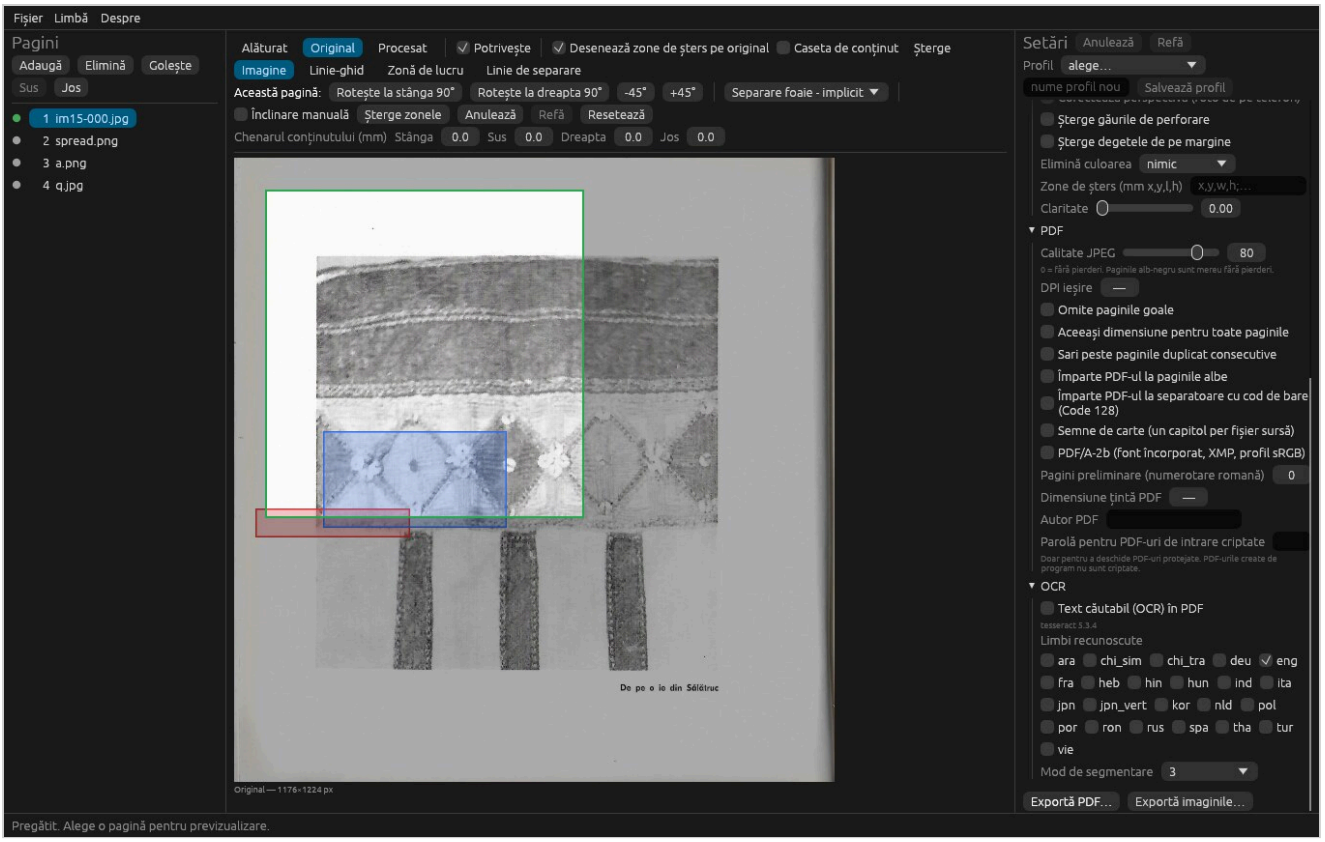
The formats of the text fields

These fields exist in the settings, but it is easier to draw on the original (section 4):

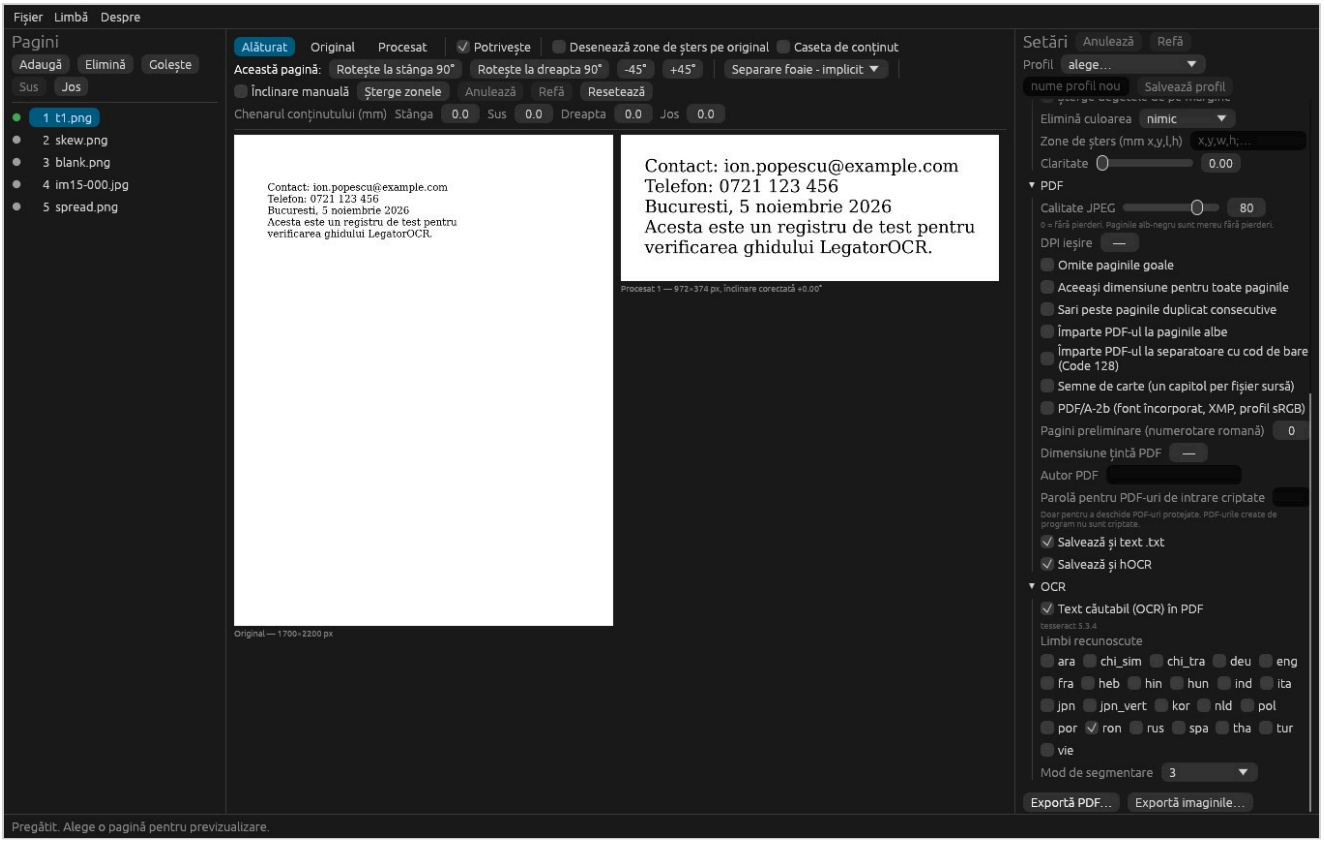
- **Erase zones** and **Picture zones**: `x,y,w,h` in **millimetres** from the top-left corner of the page. Several zones are separated with semicolons: 10,20,30,40;50,60,20,20. At most 64; width and height must be positive. If the form is wrong, the field turns red.
- **Straightening guide lines**: `x,y x,y x,y;x,y x,y x,y`, with **fractions between 0 and 1** of the page already straightened (0,0 = top-left corner). Each line has at least two points; at most 64 lines in total.
- On the command line: `--erase X,Y,W,H` (repeatable) and `--dewarp-guide "X,Y X,Y …"` (repeatable).

7. OCR: engines, languages and accuracy

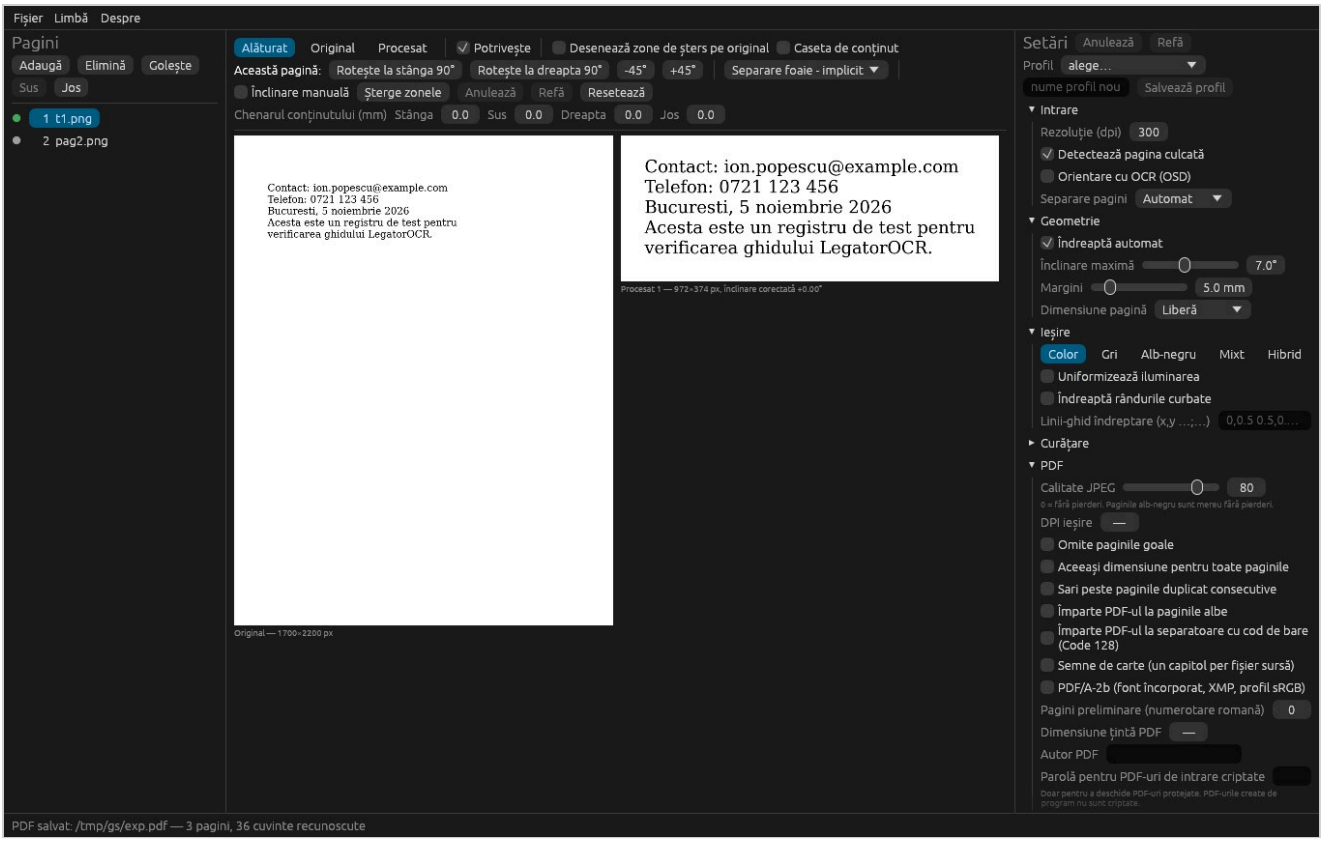
The "Searchable text (OCR) in PDF" tick box (or `--ocr` on the command line) adds an invisible text layer over the image. The language is chosen from "Recognised languages" (or `--lang ron`); it can be combined, for example `ron+eng`. The languages of the interface and of the built-in engine: Romanian, English, French, Italian, Spanish, Portuguese, German, Hungarian.



The OCR section of the settings: the "Searchable text (OCR) in PDF" tick box, the recognised languages (those installed in Tesseract) and the segmentation mode. In the screenshot Tesseract 5.3.4 was installed, which is why all its languages appear; I used the ron choice in the next figure.



With OCR ticked, "Also save .txt text" and "Also save hOCR" appear. Here the chosen language is ron.



After the export, the bottom bar shows: "PDF saved: ... — 3 pages, 36 words recognised". (I started the export with a test mechanism of the program, not from the file dialog.)

Three engines

Engine	How you choose it	Notes
Native (the program's own)	`--engine native`	no installation, written in Rust; model trained on synthetic text, then adjusted on lines from a Romanian magazine and from a 1960 textbook
Tesseract	`--engine tesseract`	separate program, see section 2; used automatically when installed and has the requested language (`--engine auto`, the default)
ONNX bridge	`--engine tesseract --tesseract scripts/engine-onnx.py`	for advanced users; requires Python, onnxruntime and separately downloaded models

```
$ legatorocr scans/text.png --engine native --ocr --lang ron --txt nat.txt --format none; cat nat.txt; echo; legatorocr
scans/text.png --engine tesseract --ocr --lang ron --txt tes.txt --format none; cat tes.txt
OCR engine: LegatorOCR native engine 0.2.0-pre.19, language: ron
text: 972x374 px @ 300 dpi, skew +0.00°
wrote nat.txt
Contact: ion.popescu(aexample.com
Telefon: 0721 123 456
Bucuresti, 5 noiembrie 2026
Acesta este un registru de test pentru
verificarea ghidului LegatorOCR.
OCR engine: tesseract 5.3.4, language: ron
text: 972x374 px @ 300 dpi, skew +0.00°
wrote tes.txt
Contact: ion.popescuGexample.com
Telefon: 0721 123 456
Bucuresti, 5 noiembrie 2026
Acesta este un registru de test pentru
verificarea ghidului LegatorOCR.
```

The same test page read by the two engines. Both made the same mistake: "@" came out as "(a" with the native engine and as "G" with Tesseract.

The measured figures (and what they do not say)

8 good pages from a Romanian magazine were measured, against a reference text produced by ABBYY, not corrected by a person. The character error rate, with diacritics (the original measurement, from edition 0.2.0-pre.21; on a new measurement in pre.26 on the same 8 pages, with the same method, the native engine gave 2.76 %; I have not investigated the difference from 2.55 %):

Engine	Character errors
Native	2.55 %
Tesseract (`ron`)	4.32 %
OCRmyPDF (with Tesseract)	4.62 %

Read simply: the native engine makes fewer mistakes **on this type of print**. But it was tuned on text from the same sources, so it cannot be said to be better in general. On other documents, especially old or unusual print, it may be weaker than Tesseract. Recognition of whole words is almost equal between the engines (about 92 %).

Accurate mode and old orthography

- `--accurate` (native engine): each line is read three times (shifted by one pixel up and down) and the probabilities are averaged. It is slower (measured: 18 % to 74 % more time) and helped only on the magazine pages (character error 3.66 % → 3.39 % on 12 pages); on books and brochures it changed nothing measurable. It is off by default.
- The Romanian word list also accepts the orthography of 1953–1993 (`cuvînt`, `romîn`, `sînt`): on five pages from a 1960 textbook, transcribed by hand, correct characters rose from 99.6 % to 99.7 % and words from 96.0 % to 96.3 %. A small difference, on five pages.
- Words are split at wide spaces even when the network did not write a space: the cells of a table no longer come out stuck together (`CantitatePreț`). A table with lines is found as a single table (Excel: `--tables-xlsx`); a table without lines is still found as two tables (left and right), an unsolved problem.

Tips

- On your own documents, compare the engines on 5–10 pages before the batch: `scripts/compare-ocr-text.py` measures the errors against reference texts that you have.
- Pages under 250 dpi are automatically enlarged before recognition.
- Pages with illustrations or diagrams can produce meaningless text in the area of the image (Tesseract read an illustration as text on one of the measured pages). Check these areas.
- Any OCR can invent or lose words. For deeds, registers or figures, spot-check.

8. Outputs and archiving

What you want	How
PDF with searchable text	`--pdf FILE.pdf --ocr` (interface: Export PDF with OCR ticked)
PDF for archiving (PDF/A-2b)	`--pdfa`; other parts with `--pdfa-part 1` or `3`

What you want	How
Processed images	`-o FOLDER`, with `--format png`, `jpeg` or `tiff`; `--format none` writes no images
Plain text of the pages	`--txt FILE.txt`
hOCR	the "Also save hOCR" tick box
Structure for accessibility	`--tagged ro` (tagged paragraphs). PDF/UA conformance has **not** been validated
DjVu copy	`--djvu FILE.djvu`
Proof of integrity	`--checksums` and `--archive-package DIR`, then `--verify-sums`
Project (settings and corrections)	File → Save project (`.lgr`), or `--save-project`; `--embed-project` hides it in the PDF

About PDF/A

The program writes PDF/A-2b with an embedded font and sRGB profile. Conformance was checked on samples, **not** with veraPDF on every output. If an institution requires certified conformance, run your own validator on the final files.

About ScanTailor projects

`--project FILE.ScanTailor` opens a ScanTailor project: the scan list and the rotation per image are taken over, while the skew, the splitting and the content box are found again. The reader was written from the description of the format and tested on a hand-made project, not on files from every ScanTailor version.

```
$ legatorocr scans/text.png --pdf sp.pdf --save-project pr.lgr --format none --work-area 5,5,80,60 --cut-at 0.5; grep -n
 'work_area\|cut_at' pr.lgr; legatorocr --project pr.lgr --pdf sp2.pdf --format none
project saved: pr.lgr
text_L: 820x374 px @ 300 dpi, skew +0.00°
text_R: 270x354 px @ 300 dpi, skew +0.00°
wrote sp.pdf (2 page(s))
31:work_area = 5,5,80,60
32:cut_at = 0.5
text_L: 820x374 px @ 300 dpi, skew +0.00°
text_R: 270x354 px @ 300 dpi, skew +0.00°
wrote sp2.pdf (2 page(s))
```

--save-project and --project: the project is saved and reopened with the same result (2 pages). Note: in pre.18, --work-area and --cut-at were not kept in the project; fixed in pre.19 (checked: the saved and reopened project gives the same pages).

9. Common problems and known limits

Symptom	What to check
The page comes out cut off or with too much white	the content box (section 4); the frame in mm in the "This page" bar
A skewed page stays skewed	the maximum skew allowed; set the angle manually
The double-page sheet is not cut where it should be	the page splitting mode: "at the middle" or "off"; since pre.18 you can drag a "Cut line" on the original or use `--cut-at 0.45`
A text field turns red	the format of the field (section 6): millimetres for zones, fractions 0–1 for guides
The OCR text has mistakes	compare the engines (section 7); check the resolution; switch to Tesseract or the other way round

Symptom	What to check
The "To check" panel	pages marked as suspect (blank, borderline skew, nothing found, content stuck to the edge); they are not necessarily wrong, but they are worth a look
The window does not work on Windows or in RDP	missing OpenGL 2+; try the video card driver
The macOS app does not start	it is not signed: right-click → Open or the `xattr` command from section 2

Known limits of edition 0.2.0-pre.61

- Auto keystone: checked on synthetic sheets and on 67 phone photos of a single book (133 pages), where it left most pages alone and lowered the measured tilt on the rest; nobody has judged the result by eye on other books, or on pages with large pictures, tables or very short lines, where it may do nothing or too little. The settings per page and the cut-line lean were tried on synthetic sheets and in tests; the table pencil makes the OCR read the zone as one block, it does not find cells.

- Rotation by 90°: on a page with a photograph of ornaments (page 15 of a PDF) a tester on Windows saw the rotation jump from 90° to 180°. I reproduced part of the problem on Linux: in pre.18, with "Detect page lying on its side" ticked (the default), the program wrongly rotated this page by 90°, although the original is upright (the figure in section 4). The threshold in pre.19 was too fragile (I have meanwhile discovered that another page of the same PDF, page 13, was also rotated wrongly). In pre.20 the detector counts only pieces the size of a letter and abstains on pages with pictures or with few words. On 98 book pages (rotated at 0° and 90°) and 5 pages with pictures, the result is identical to pre.18, apart from page 13 (rotated wrongly in pre.18, now upright) and one page with a photo lying on its side, which the detector no longer turns (OSD and manual rotation remain). Checked on the command line, on Linux, on a small corpus. The probable cause of the jump (automatic orientation added on top of the manual rotation) was fixed in pre.18: after a click on rotate the page stays locked, and four clicks give 90°, 180°, 270° and back to 0° (checked in the window, on Linux). What exactly the tester saw on Windows 11 I could not check. If a page with pictures is still rotated wrongly, untick "Detect page lying on its side" or use --no-auto-orient.

- The work area, the movable cut line, the erase and picture zones, the grid of points, Undo and Ctrl+Z, the content box, OCR and PDF export were tried in a real window on Linux, with a simulated mouse (the screenshots in the guide). Resizing a zone from a corner worked in the window. The native file dialogs ("Export PDF…", "Save project", "Open project") worked with the GTK dialog of an xdg portal on Linux; on Windows and macOS I did not try them. If the system has no dialog at all, since pre.21 the window says so instead of doing nothing (you can drag the files into the window or use the command line). Saving a custom profile worked. The effect of the cut line I checked through automated tests, with --cut-at and in the window (a double sheet cut into two unequal pages, 524 and 674 px). For sheets rotated by 90° or 270° (since pre.21) the pages come out one below the other; this is checked only through an automated test, not in the window.

  --save-project: in the published build pre.18, --work-area and --cut-at given on the command line were not kept in the .lgr file (found while checking this guide). Fixed in pre.19. If you use pre.18, give the options again with --project, not only from the project.

- The graphical interface has not been checked by people on Windows, macOS or arm64.

- PDF/UA verification, the CIUS-RO electronic invoice and the time stamp over HTTPS with a real authority have not been validated.
- For a certified PDF/A copy, the validator is yours (for example veraPDF).
- The right to use the program's own OCR model (partly trained on pages with unclarified rights) is an open matter for the author; see `docs/audit/assets-licences.md`.
- Automatic upside-down detection: Latin-script text only, at least 12 lines per page; a page with tables or little text may be left alone (use “180°”). The collections window and the large thumbnails have automated tests, but the author has not seen them on screen.

10. How to report a problem

The most useful contribution is a report from a system the author has not tried. It takes five minutes:

1. Download the latest edition for your system.
2. Run `legatorocr --doctor` and copy the whole output.
3. Run a page you can share: `legatorocr PAGE.jpg --ocr --pdf result.pdf -o pages`.
4. Open `result.pdf`: is the page upright and cropped as it should be? Can the text be selected and searched?

Send, as an issue in the `chiuta/LegatorOCR` repository (or by e-mail):

- the system and its version (for example Windows 11 23H2, macOS 15 arm64, Debian 12 arm64);
- the LegatorOCR version;
- the output of the `--doctor` command;
- the command you ran (or the setting in the interface);
- what you expected and what happened (error text or screenshot);
- whether the image can be shared.

Do not send documents with personal data. An "everything worked" report is still useful: it is the only way the author learns that a system works.

Security problems are reported according to `SECURITY.md`, not publicly.